

Semáforos Inteligentes: Un enfoque con Aprendizaje por Refuerzo Multi Agente

Tesina de Licenciatura en Ciencias de la Computación

Juan Martín Morales

Tutora: Dra Ana Carolina Olivera



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE
INGENIERÍA

Facultad de Ingeniería
Universidad Nacional de Cuyo

Resumen

En este trabajo se aborda el problema de la congestión vehicular en áreas urbanas, proponiendo una solución basada en el Aprendizaje por Refuerzo Multi-Agente (MARL) Cooperativo para optimizar el control semafórico. Se analiza la evolución del parque automotor en Argentina, destacando el aumento significativo de vehículos y sus consecuencias en la movilidad urbana. Se presentan los desafíos asociados al control semafórico tradicional y se justifica la necesidad de enfoques adaptativos basados en inteligencia artificial. El objetivo principal es desarrollar y evaluar un sistema de control de tráfico que permita mejorar el flujo vehicular mediante políticas coordinadas entre múltiples intersecciones, utilizando simulaciones tanto en escenarios sintéticos como en la red vial real del Gran Mendoza. **Caro: falta completar**

Abstract

In this work, we address the problem of vehicular congestion in urban areas by proposing a solution based on Cooperative Multi-Agent Reinforcement Learning (MARL) to optimize traffic light control. We analyze the evolution of the automotive fleet in Argentina, highlighting the significant increase in vehicles and its consequences on urban mobility. We present the challenges associated with traditional traffic light control and justify the need for adaptive approaches based on artificial intelligence. The main objective is to develop and evaluate a traffic control system that improves vehicular flow through coordinated policies among multiple intersections, using simulations in both synthetic scenarios and the real road network of Greater Mendoza. **Caro: falta completar**

Agradecimientos

Página de agradecimientos

Índice

Resumen	1
1 Introducción	2
1.1 Motivación	3
2 Marco Teórico	4
2.1 Ingeniería de Tránsito y Simulación	4
2.1.1 Terminología de Tránsito	4
2.1.2 Fundamentos de la Dinámica Vehicular	6
2.1.3 Sistemas de Control de Tránsito	9
2.1.4 Simulación de Tránsito	11
2.2 Aprendizaje por Refuerzo (RL)	14
2.2.1 El Proceso de Decisión de Markov (MDP)	14
2.2.2 Métodos Basados en Valor y Basados en Política	15
2.2.3 Aprendizaje Profundo y Redes Neuronales	16
2.2.4 Aprendizaje por Refuerzo Profundo (DRL)	18
2.3 Aprendizaje por Refuerzo Multi-Agente (MARL)	21
2.3.1 Fundamentos Teóricos: Juegos Estocásticos, Equilibrio de Nash y Dec-POMDP	21
2.3.2 Desafíos Fundamentales en MARL	22
2.3.3 Paradigmas de Entrenamiento y Ejecución	23
2.3.4 Multi-Agent Deep Reinforcement Learning (MADRL) y Algoritmos Baseline	23
2.3.5 Multi-Agent Proximal Policy Optimization (MAPPO)	24
2.3.6 Heterogeneous-Agent Proximal Policy Optimization (HAPPO)	26
2.4 Control de Tráfico Urbano mediante MARL	30
2.4.1 Formalización del Control Semafórico Coordinado como Dec-POMDP	30
2.4.2 Funciones de Recompensa en Control Semafórico	31
2.5 Frameworks Tecnológicos de Escalado y Optimización	32
2.5.1 Ecosistema Ray: Ray Core, Ray Tune y RLlib	32
2.5.2 Framework Optuna y Pipeline de Optimización Multietapa	32
3 Metodología y Diseño Experimental	34
3.1 Enfoque y Diseño de Investigación	34
3.2 Especificación del Stack Tecnológico	35
3.3 Diseño de Escenarios de Tránsito en SUMO	36
3.3.1 Escenarios Sintéticos: 3×1 , 3×3 y $4 \times 4_H$	36
3.3.2 Escenario Urbano Real: Gran Mendoza (Corredor Vicente Zapata)	37
3.4 Arquitectura de Integración Entorno-Agentes	37
3.4.1 Ciclo Episódico e Interfaz de Simulación	37
3.5 Instanciación del Modelo Dec-POMDP	37
3.5.1 Espacio de Observaciones y Codificación Logarítmica	38
3.5.2 Espacio de Acciones y Enmascaramiento Dinámico	38
3.5.3 Función de Recompensa Compuesta y Coordinación Vecinal	38
3.6 Implementación Algorítmica e Hiperparámetros	39
3.7 Diseño de Experimentos de Búsqueda Hiperparamétrica	39
3.7.1 Infraestructura Computacional de Alto Rendimiento y Orquestación Distribuida	39
3.8 Diseño de Experimentos de Evaluación de Rendimiento	40

4	Análisis y discusión de resultados	41
4.1	Resultados del ajuste de hiperparámetros	41
4.1.1	Diseño del proceso de ajuste	41
4.1.2	Resultados del ajuste y análisis de sensibilidad	41
4.1.3	Configuración seleccionada para la evaluación final	41
4.2	Resultados de los modelos finales	41
4.2.1	Criterios de evaluación y métricas de comparación	41
4.2.2	Desempeño en escenarios sintéticos (3×1 , 3×3 , 4×4 heterogéneo) . . .	41
4.2.3	Desempeño en la red de carreteras de la Ciudad de Mendoza	41
4.3	Discusión e interpretación de resultados	41
5	Conclusiones	42

Índice de códigos

Índice de figuras

1.1	Evolución de la flota vehicular circulante en Argentina (1990–2025). Datos: AFAC.	2
2.1	Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de FHWA [Fed08].	5
2.2	Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.	5
2.3	Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).	7
2.4	Comparación esquemática de la distribución de intervalos temporales entre vehículos en flujo libre (exponencial negativa, consistente con el proceso de Poisson) y en flujo coordinado por semáforos (distribución desplazada con zona de exclusión física $t_{\min}$ y pico característico de pelotón).	9
2.5	Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.	10
2.6	Elementos geométricos y conexiones de una intersección semaforizada en SUMO bajo sentido de circulación por la derecha (Argentina): carriles entrantes (L_{in}), salientes (L_{out}), líneas de detención y enlaces de cruzamiento (<i>links</i>) en verde prioritario ($\mathbf{G}$) y rojo retenido en conflicto ($\mathbf{r}$).	12
2.7	Interacción agente-entorno en un proceso de decisión de Markov [SB18].	15
2.8	Taxonomía de algoritmos de Aprendizaje por Refuerzo, enfocada en métodos <i>Model-Free</i> (adaptada de OpenAI <i>Spinning Up</i> [Ach18]).	16
2.9	Interacción Agente-Entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.	22
2.10	Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). El Crítico utiliza información global privileged para guiar las actualizaciones de los Actores locales durante la simulación.	24
2.11	Esquema de actualización secuencial de HAPPO. A diferencia de PPO Independiente, la actualización de cada agente está matemáticamente condicionada por el cociente de probabilidad compuesto M_{i_m} , que propaga los cambios efectuados por los agentes que lo precedieron en la secuencia.	29
3.1	Flujo metodológico secuencial de la investigación experimental.	35

Glosario

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso exclusivo a una o más corrientes vehiculares incompatibles para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

Resumen

Capítulo de resumen

Ejemplo de items

- Item 1
- Item 2

Introducción

En las últimas décadas, en el mundo se ha producido un incremento sostenido del parque automotor a nivel mundial, acompañado por un proceso continuo de urbanización que ha convertido a la congestión vehicular en uno de los principales desafíos para la movilidad y eficiencia en áreas urbanas. Según los reportes de flota vehicular de Argentina, elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y utilizando el año 2012 como línea base, el volumen de vehículos registró un aumento del 21,7% en 2017. Dicha progresión se sostuvo a lo largo de la última década, alcanzando un incremento acumulado del 44,4% para el año 2025 (véase la Figura 1.1).

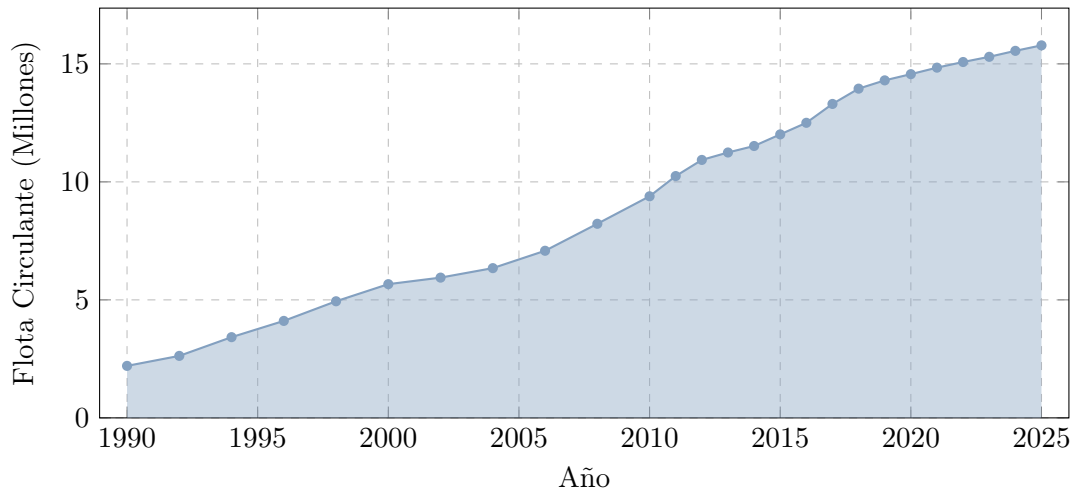


Figura 1.1: Evolución de la flota vehicular circulante en Argentina (1990–2025). Datos: AFAC.

Las congestiones vehiculares traen consigo múltiples consecuencias: desde importantes pérdidas de productividad por demoras en los tiempos de viaje, hasta un aumento en la cantidad de detenciones vehiculares y la consecuente emisión de gases nocivos para la salud y el medio ambiente, tales como CO , CO_2 , NO_x , HC [ne18]. Este escenario impacta de manera notable en grandes ciudades con una alta densidad de vehículos. En este trabajo se utilizaron casos de estudio sintetizados y como caso de estudio real el Área Metropolitana del Gran Mendoza (GM), comprendida entre Avenida San Martín, Avenida Colón, Avenida Belgrano y Avenida Las Heras.

Frente a un escenario de congestión, una alternativa viable, sin recurrir a reestructuraciones costosas en la infraestructura, consiste en optimizar la gestión del tránsito mediante sistemas de control semafórico adaptativos. Los enfoques tradicionales de control semafórico han demostrado poca capacidad de adaptación ante escenarios con mucha variabilidad de flujos de vehículos. Una alternativa prometedora resulta ser el Aprendizaje por Refuerzo (*Reinforcement Learning*, RL), un paradigma de inteligencia artificial donde los semáforos pueden modelarse como agentes, capaces de aprender políticas de control a partir de la interacción con un entorno (la situación del tránsito) mediante la ejecución de acciones (cambios de fase) con el objetivo de maximizar una recompensa (como, por ejemplo, el valor negativo de la suma de los tiempos de espera). Diversos estudios han demostrado la viabilidad de este enfoque, evolucionando desde la aplicación de algoritmos clásicos y tabulares en una sola intersección [Wie00, APK03], hasta modelos a gran escala de redes coordinadas empleando redes neuronales profundas y control multi-agente [ETAA13, LLW16, W⁺19, CWCL19].

Sin embargo, aplicar este enfoque de manera simultánea sobre múltiples intersecciones introduce desafíos adicionales. Un esquema de control centralizado resulta poco escalable debido al crecimiento exponencial del espacio de acciones conjunto. Una alternativa “ingenua” del Apre-

dizaje por Refuerzo Multi-Agente resulta ser el Aprendizaje Independiente (*Independent Learning* o IL), donde cada semáforo busca descongestionar de forma independiente y egoísta solo su propia intersección [AS22]. Este enfoque tiene dos desventajas críticas: primero, la optimización puramente local conduce a un óptimo global sub-óptimo; segundo, dado que todos los agentes aprenden y cambian sus políticas simultáneamente, el entorno se vuelve no estacionario para cada agente individual, lo cual viola los supuestos de convergencia del aprendizaje [FH21].

Por esta razón, en este trabajo final de carrera se aborda el problema desde el marco del Aprendizaje por Refuerzo Multi-Agente (MARL) Cooperativo. Bajo este paradigma, los controladores (agentes) actúan como un equipo. Aprenden políticas coordinadas, no para maximizar solo una recompensa individual, sino también una recompensa global compartida orientada a la minimización del tiempo de espera promedio de toda la red, evitando así los conflictos que surgen del aprendizaje independiente. La recompensa es un valor numérico que indica cuán efectiva fue la acción conjunta.

En este contexto, el objetivo general de este trabajo fue desarrollar y evaluar un sistema de control de tráfico basado en este enfoque cooperativo para optimizar el flujo vehicular urbano. Para lograrlo, se analizó la convergencia a políticas de control óptimas y la escalabilidad de distintos algoritmos MARL ante entornos no estacionarios mediante simulaciones en escenarios sintéticos y en la red vial real del Gran Mendoza. Esto permitió establecer una línea base de comparación para, posteriormente, validar empíricamente el desempeño de la arquitectura descentralizada y coordinada propuesta frente a los enfoques de control tradicionales y a metodologías recientes de la literatura.

1.1 Motivación

La motivación principal de este trabajo radica en la marcada asimetría entre una demanda vehicular en constante crecimiento y una infraestructura vial que permanece estática. Dado que ensanchar avenidas o modificar físicamente el trazado urbano en cascos densos como el del Gran Mendoza es económica y estructuralmente inviable, la alternativa más sostenible es la optimización lógica de los recursos existentes.

El desarrollo de soluciones basadas en software permite dotar de la adaptabilidad necesaria a la red semafórica actual. De este modo, es posible transformar un sistema rígido en uno inteligente capaz de mitigar las demoras y reducir la contaminación ambiental evitable, maximizando la eficiencia de la infraestructura vial sin requerir obras civiles de gran envergadura.

La presente tesina se organiza de la siguiente manera: en el Capítulo 1 se formula el problema de estudio y se exponen las motivaciones que impulsan esta investigación. A continuación, el Capítulo 2 establece las bases teóricas, abordando los conceptos clave de la ingeniería de tránsito y los fundamentos del *Reinforcement Learning*, particularmente en su vertiente multi-agente. Luego, el Capítulo 3 detalla la metodología de trabajo, describiendo tanto los entornos de simulación empleados como la arquitectura algorítmica de los modelos implementados. Posteriormente, en el Capítulo 4 se lleva a cabo un análisis empírico del desempeño de los distintos controladores, evaluándolos en escenarios controlados y en la red urbana simulada del Gran Mendoza. Finalmente, el Capítulo 5 sintetiza los hallazgos principales, destaca las contribuciones metodológicas y sugiere posibles vías para continuar este trabajo en el futuro.

Marco Teórico

2.1 Ingeniería de Tránsito y Simulación

La ingeniería de tránsito se define como aquella fase de la ingeniería de transporte —disciplina que se enmarca tradicionalmente dentro de la ingeniería civil— que se ocupa de la planeación segura y eficiente, el proyecto geométrico y la operación del tránsito por calles y carreteras, sus redes, terminales, tierras adyacentes y su relación con otros modos de transporte [CyMR18]. Asimismo, el *Traffic Engineering Handbook* la describe formalmente como la subdisciplina enfocada en la planificación, el diseño geométrico y la operación de redes viales [PW16]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa: permiten describir el funcionamiento físico de la red vial, caracterizar la oferta y demanda de circulación, e identificar las variables operacionales sobre las cuales puede intervenir un sistema de control inteligente.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en proveer y ampliar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y geométricamente inviable [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

2.1.1 Terminología de Tránsito

Antes de introducir los mecanismos de control y su formulación computacional, conviene precisar algunos términos operativos que se utilizarán de manera recurrente en este documento. Esta aclaración inicial es habitual en manuales de temporización semafórica, donde se advierte que el término fase puede confundirse con movimiento si no se distingue la unidad de control del conjunto de trayectorias servidas [Fed08], y en estudios de control semafórico basados en aprendizaje por refuerzo, donde los conceptos de movimientos, fases y señales se explicitan antes de formular el problema de decisión [W⁺19].

En este contexto, se emplearán las siguientes nociones:

- **Intersección, acceso y carril:** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].
- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una

salida determinada (e.g., giros o movimientos directos) [W⁺19]. Dos movimientos se consideran en conflicto o incompatibles cuando sus trayectorias se intersectan y generan puntos de conflicto en la intersección (véase la Figura 2.2), lo que impide su servicio simultáneo por razones de seguridad [FA11].

- **Fase, ciclo e intervalo:** una fase representa el estado operativo del semáforo que otorga el derecho de paso a un grupo de movimientos compatibles de manera simultánea [PW16]. La secuencia completa de fases requerida para servir a todos los movimientos de la intersección constituye el ciclo semafórico, mientras que un intervalo es la fracción temporal del ciclo durante la cual las indicaciones del semáforo no varían [CyMR18].

La Figura 2.1 muestra una representación típica de estos conceptos para una intersección de cuatro accesos. La convención utilizada por manuales de temporización como el de la FHWA asigna números impares a los giros izquierdos y números pares a los movimientos directos; los giros derechos suelen operar junto con el movimiento directo compatible, mientras que los cruces peatonales se sirven de manera concurrente con la corriente vehicular paralela [Fed08]. Esta representación no fija una geometría particular para el caso de estudio, sino que permite distinguir entre movimiento físico, fase semafórica y usuarios servidos por una misma indicación.

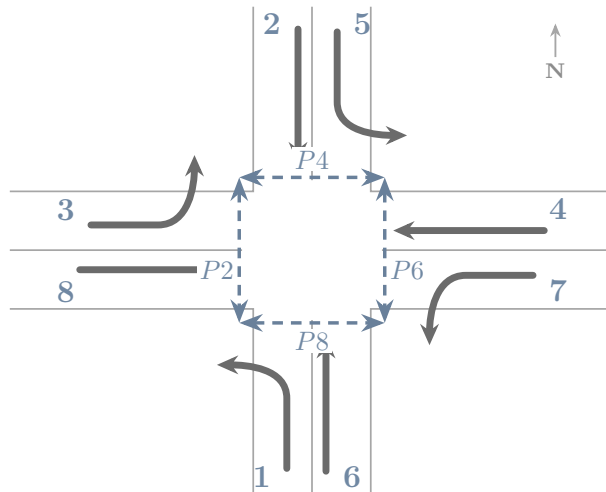


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de FHWA [Fed08].



Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí. Por ello, su

operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2 Fundamentos de la Dinámica Vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flujo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora (veh/h), calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas [CyMR18].
- **Densidad o concentración (k):** indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro (veh/km), calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado [CyMR18]. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].
- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial [CyMR18].
- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado [CyMR18].

En el nivel microscópico, las variables asociadas describen el movimiento del vehículo individual y la separación entre vehículos consecutivos (véase la Figura 2.3):

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.

- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos (e.g., parachoques delanteros) [CyMR18]. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.
- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo [CyMR18]. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

Cuadro 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	Variable Principal	Relación / Conversión
Macroscópica	Flujo (q , veh/h)	$q = 3600/h_{\text{prom}}$
	Densidad (k , veh/km)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s , km/h)	$q = k \cdot v_s$
	Velocidad media temporal (v_t , km/h)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i , km/h)	Variable individual de cada vehículo
	Intervalo temporal (h , s)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s , m)	Distancia espacial entre parachoques homólogos

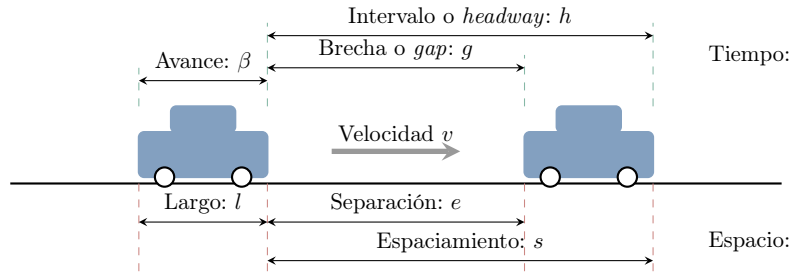


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).

Estas variables macroscópicas y agregadas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \quad (2.1)$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. Cabe destacar que la velocidad media espacial (v_s) es la única físicamente consistente con esta ecuación fundamental. La velocidad media temporal (v_t) difiere de esta debido a que asigna mayor peso a los vehículos más rápidos que cruzan el punto de medición en un lapso dado. La vinculación analítica entre ambas variables se establece formalmente mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \quad (2.2)$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. Dado que la varianza es siempre no negativa ($\sigma_s^2 \geq 0$), se cumple que $v_t \geq v_s$, igualándose únicamente en el caso hipotético en que todos los vehículos circulen exactamente a la misma velocidad. En intersecciones semaforizadas, esta relación dinámica permite modelar cómo las interrupciones del semáforo durante la luz roja detienen el flujo ($v_s \rightarrow 0$), provocando una acumulación de vehículos que eleva la densidad ($k \rightarrow k_{\max}$) y genera colas y demoras asociadas a condiciones de flujo forzado [CyMR18].

Métricas de Ineficiencia Operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** Representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.
- **Colas:** Corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de ser servidos. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección aguas arriba y propagar la congestión al resto de la red.
- **Detenciones:** Cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Descripción Probabilística del Flujo y Pelotones Vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \quad (2.3)$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \quad (2.4)$$

donde h es el intervalo temporal en segundos, q es el flujo en veh/h, y t es el umbral de tiempo analizado.

Esta representación probabilística asume independencia entre arribos sucesivos, siendo una aproximación válida únicamente para flujos vehiculares bajos o moderados bajo condiciones de flujo libre, donde no exista la influencia de semáforos cercanos aguas arriba [CyMR18]. Sin embargo, este supuesto de independencia decae en condiciones de congestión o en redes viales urbanas semaforizadas. En estos casos, la operación de los semáforos interrumpe periódicamente el flujo y agrupa a los vehículos en pelotones (*platoons*), induciendo una fuerte dependencia espacio-temporal y correlación entre arribos consecutivos. La Figura 2.4 presenta una comparación conceptual de la distribución resultante de los intervalos temporales entre vehículos bajo estas dos condiciones operativas.

Esta limitación de los modelos analíticos tradicionales (como los de Poisson o teoría de colas clásica) justifica la necesidad de recurrir a la simulación microscópica de tránsito. En un simulador como SUMO, la formación, dispersión e interacción de pelotones vehiculares emergen de manera natural a partir de los modelos de seguimiento de vehículos y cambio de carril, permitiendo evaluar el control semafórico bajo flujos dinámicos realistas. Asimismo, incorporar arribos estocásticos no Poissonianos, perfiles de demanda variables y la inicialización de múltiples corridas con diferentes semillas aleatorias en el simulador es un procedimiento estándar para evaluar la robustez y capacidad de adaptación de cualquier esquema de control dinámico ante la incertidumbre operativa [CyMR18]; esto permite asegurar que los algoritmos sean capaces de responder ante fluctuaciones dinámicas del tráfico en tiempo real, en lugar de optimizarse para una secuencia de flujo determinista y artificial [PW16].

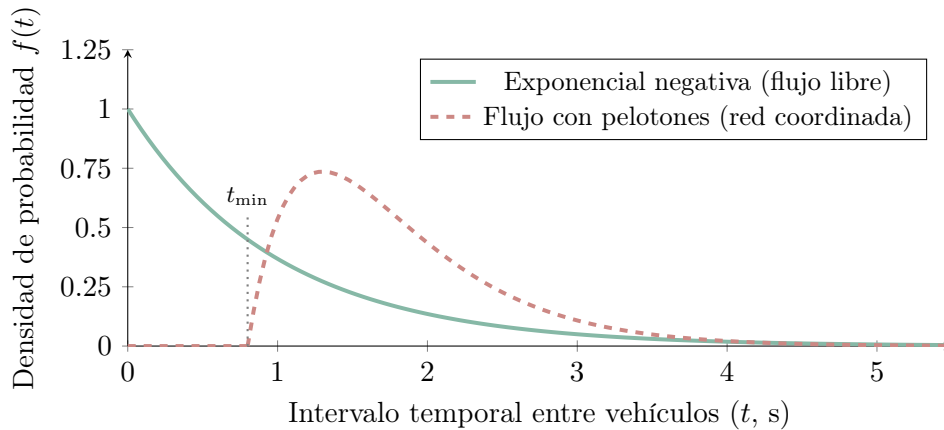


Figura 2.4: Comparación esquemática de la distribución de intervalos temporales entre vehículos en flujo libre (exponencial negativa, consistente con el proceso de Poisson) y en flujo coordinado por semáforos (distribución desplazada con zona de exclusión física $t_{\min}$ y pico característico de pelotón).

2.1.3 Sistemas de Control de Tránsito

Para mitigar los conflictos de tránsito e incrementar la seguridad en las intersecciones urbanas de flujo interrumpido, se emplean sistemas de control de tránsito [PW16]. El dispositivo de control por excelencia en estos nodos es el semáforo, definido como un dispositivo electrónico que regula la circulación de vehículos asignando el derecho de paso alternado mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelado algorítmico, los semáforos y sus parámetros de configuración constituyen los elementos actuadores de la red. La modificación de la duración del verde efectivo, la secuencia de fases y la sincronización entre nodos define el conjunto de intervenciones dinámicas mediante las cuales el sistema de control busca optimizar el paso vehicular.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje.

Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. Para el presente trabajo, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semafórico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la Capacidad y el Verde Efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la práctica, la descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control, como se ilustra conceptualmente en la Figura 2.5. En este trabajo, estos efectos no se modelan mediante fórmulas analíticas de capacidad, sino mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

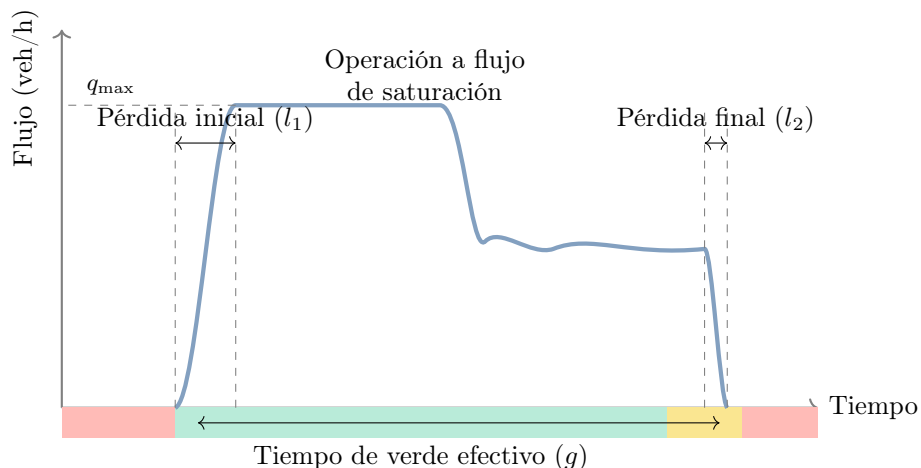


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

Clasificación de los Controladores de Semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo,

la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite de seguridad y funcionamiento, como el verde mínimo ($g_{\min}$) y el verde máximo ($g_{\max}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4 Simulación de Tránsito

Debido a la densidad y complejidad de la red urbana moderna, la sincronización y optimización de los tiempos de semáforos no pueden resolverse adecuadamente con modelos estáticos bajo condiciones dinámicas, por lo que se recurre a aproximaciones computacionales basadas en simulación de tránsito [PW16]. De acuerdo con la taxonomía clásica [FA11], estos modelos pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los modelos mesoscópicos representan entidades intermedias, como grupos o pelotones de vehículos. Ambos niveles son útiles para estudiar fenómenos de red, pero no ofrecen el detalle necesario para observar cómo las decisiones semafóricas afectan, paso a paso, la formación de colas y la interacción local entre vehículos en una intersección.

Para el presente trabajo, el nivel de mayor interés es el microscópico, ya que representa explícitamente a cada vehículo, su ruta y su movimiento dentro de la red [FA11]. SUMO (*Simulation of Urban MObility*) se ubica en esta categoría: es un simulador de tránsito microscópico, multimodal y de código abierto, diseñado para representar demandas vehiculares sobre redes viales y evaluar problemas de gestión del tránsito [ALBBW⁺18, DLR25]. En SUMO, la evolución vehicular es continua en el espacio y discreta en el tiempo; además, el entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR25]. Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio no se evalúan mediante una fórmula cerrada de demora, sino observando sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles y propagación de congestión. Por ello, la simulación microscópica constituye el puente natural entre el problema de tránsito y su formulación computacional, al permitir que el controlador interactúe con un entorno dinámico y medible durante la ejecución.

Modelos Microscópicos de Comportamiento Vehicular

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. En SUMO, una intersección semaforizada se estructura mediante tres componentes geométricos y lógicos principales bajo el sentido de circulación por la derecha (véase la Figura 2.6) [DLR25]:

- **Carriles entrantes (L_{in}) y salientes (L_{out}):** Los carriles entrantes conducen el flujo vehicular hacia la línea de detención de la intersección por la derecha de cada acceso, mientras que los carriles salientes reciben el flujo descargado tras cruzar el nodo.
- **Líneas de detención (*stop lines*):** Delimitación física continua en color blanco en el límite de los carriles entrantes L_{in} con el área interna de cruzamiento (*junction*).
- **Enlaces de conexión (*links*):** Rutas internas que conectan un carril entrante específico con un carril saliente de destino. A cada enlace se le asigna un estado semafórico instantáneo (G para verde prioritario, g para verde permisivo, y r para rojo o detención). Como se ilustra en la Figura 2.6, mientras los movimientos directos se encuentran habilitados en verde (G),

el giro a la izquierda en conflicto (representado por el enlace en rojo salmón) se mantiene retenido en luz roja (r) para prevenir colisiones.

Esta representación es compatible con la formulación algorítmica del problema, donde una acción consiste en seleccionar una fase que habilita un conjunto de enlaces compatibles mientras mantiene en rojo (r) los movimientos en conflicto.

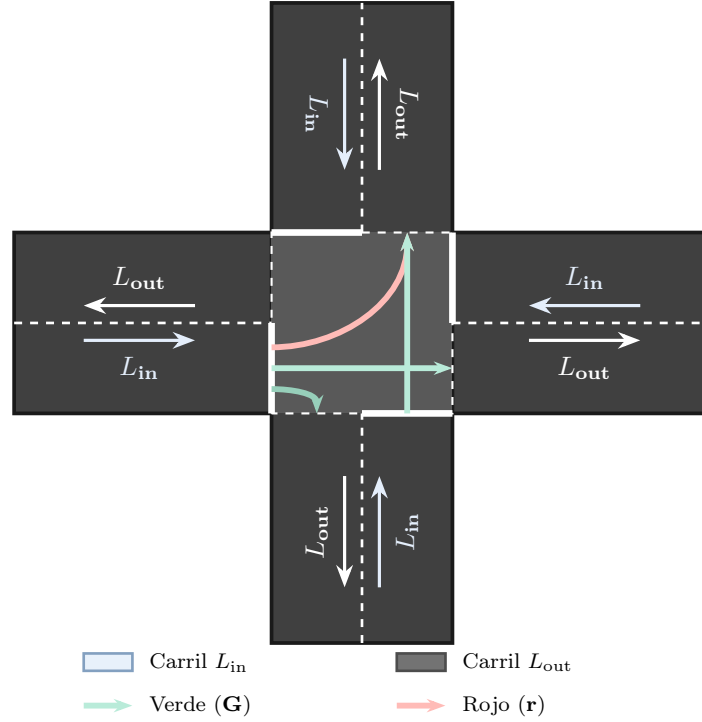


Figura 2.6: Elementos geométricos y conexiones de una intersección semaforizada en SUMO bajo sentido de circulación por la derecha (Argentina): carriles entrantes (L_{in}), salientes (L_{out}), líneas de detención y enlaces de cruzamiento ($links$) en verde prioritario (G) y rojo retenido en conflicto (r).

Componentes y Configuración de SUMO

Para la generación, edición y control en tiempo real de los escenarios de simulación microscópica, el entorno SUMO se complementa con un conjunto integrado de herramientas y librerías auxiliares [ALBBW⁺18, Ecl25]:

- **netedit**: Herramienta gráfica que permite proyectar y editar la geometría y la lógica operacional de la red vial de forma interactiva. Facilita la creación y modificación bidireccional de nodos, accesos, carriles, restricciones de giro, conexiones internas ($links$), sensores inductivos y programas de temporización semafórica.
- **osmwebwizard**: Herramienta automatizada con interfaz web que permite importar áreas geográficas reales desde OpenStreetMap (OSM) y transformarlas al formato nativo de SUMO, extrayendo la topología urbana, los límites de velocidad y perfiles sintéticos de demanda vehicular.
- **TraCI (Traffic Control Interface)**: Protocolo de comunicación cliente-servidor mediante sockets TCP que habilita la interacción dinámica a paso discreto entre la simulación y scripts externos de control [DLR25]. A través de TraCI, los agentes de aprendizaje por refuerzo recolectan observaciones del estado del tránsito (colas, velocidades, ocupación) e imponen decisiones sobre las fases del semáforo.

Desde la perspectiva del modelado computacional, la ejecución de una simulación microscópica en SUMO requiere una especificación mínima de archivos de configuración en formato XML [Ecl25], cuyos componentes principales se resumen en la Tabla 2.2:

- **Archivo de red (.net.xml):** Define la estructura física e infraestructura de la red vial, incluyendo las coordenadas nodales, la geometría de arcos, los carriles por acceso, las conexiones de cruzamiento y la lógica de accionamiento semafórico.
- **Archivo de rutas (.rou.xml):** Especifica las características de la flota vehicular (dimensiones, aceleración, velocidad deseada y modelo de seguimiento) junto con los itinerarios o flujos de origen a destino.
- **Archivo TAZ (.taz.xml, *Traffic Analysis Zones*):** Delimita las zonas de análisis de tráfico para asignar espacialmente las matrices de origen y destino de los desplazamientos.
- **Archivo de elementos adicionales (.add.xml):** Incorpora dispositivos de medición (como detectores de bucle inductivo E1 y E2), paradas de transporte público o programas semafóricos personalizados.
- **Archivo maestro de configuración (.sumocfg):** Integra las referencias a los archivos de red, rutas y adicionales, y fija los parámetros del horizonte temporal de simulación (duración del paso Δt , tiempos de inicio y fin, e indicadores de salida).

Cuadro 2.2: Requerimientos mínimos de archivos de configuración para la simulación en SUMO [Ecl25].

Archivo XML	Extensión	Función en el Modelo Computacional
Red Vial	.net.xml	Topología física, accesos, carriles, enlaces y semáforos.
Rutas y Demanda	.rou.xml	Tipos de vehículos, modelos de seguimiento e itinerarios.
Zonas TAZ	.taz.xml	Zonas de origen y destino para matrices de demanda.
Elementos Adicionales	.add.xml	Sensores inductivos E1/E2, telemetría y programas auxiliares.
Configuración Maestra	.sumocfg	Unificación de archivos, resolución temporal (Δt) y registros.

Métricas de Evaluación Operacional y Aprendizaje

Para evaluar rigurosamente el impacto de las estrategias de control semafórico tanto desde la perspectiva de la ingeniería de tránsito como del aprendizaje automático, este trabajo distingue cuatro métricas fundamentales de evaluación:

- **Emisión de Gases CO₂:** Cuantifica el impacto ambiental y la ineficiencia energética del flujo vehicular. En SUMO, las emisiones se estiman mediante modelos de emisión microscópicos (como HBEFA o PHEMlight [DLR25]), relacionando instantáneamente la masa de CO₂ emitida en gramos con los perfiles de velocidad, aceleración y tiempo en ralentí (*idling*) de cada vehículo. Una reducción en la masa total de CO₂ refleja mayor fluidez y menor frecuencia de detenciones.
- **Tiempo de Espera Promedio Global y por Agente:** Métrica operacional primaria en ingeniería de tránsito [CyMR18]. El tiempo de espera cuantifica la acumulación en segundos que un vehículo permanece detenido (velocidad $v < 0.1$ m/s) antes de cruzar la intersección. Se evalúa a nivel local por cada agente i (intersección) y de forma agregada a nivel global de la red vial, permitiendo medir la reducción directa de la demora operacional.

- **Recompensa Acumulada Descontada:** Métrica interna de RL/MARL definida como el retorno episódico $G_0 = \sum_{t=0}^T \gamma^t R_t$. Expresa la capacidad del agente para maximizar de forma persistente su objetivo escalar proyectado en el tiempo [SB18].
- **Valores de Pérdida (*Loss Values*):** Indicadores de diagnóstico durante el entrenamiento en DRL/MADRL [YVV⁺22]. Incluyen la pérdida de política o actor ($\mathcal{L}^{\text{CLIP}}$), la pérdida de error cuadrático medio de la función de valor del crítico ($\mathcal{L}^{\text{VF}}$) y el término de entropía de la política ($S[\pi]$). Su seguimiento permite detectar inestabilidades del gradiente, monitorear la tasa de convergencia y prevenir el colapso prematuro de la política (*policy collapse*).

2.2 Aprendizaje por Refuerzo (RL)

Dentro del aprendizaje automático suelen distinguirse tres paradigmas principales. En el aprendizaje supervisado, el modelo se entrena a partir de ejemplos etiquetados que indican la salida esperada para cada entrada. En el aprendizaje no supervisado, en cambio, se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El aprendizaje por refuerzo (*Reinforcement Learning*, RL) aborda un problema distinto: una entidad que toma decisiones debe aprender, mediante interacción con un entorno, qué acciones elegir para maximizar una señal de recompensa acumulada a largo plazo [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el *agente* (*agent*), que aprende y toma decisiones, y el *entorno* (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso temporal, el agente observa una situación del entorno, selecciona una acción y recibe como respuesta una nueva situación junto con una recompensa. El comportamiento del agente está determinado por una *política* (*policy*), denotada por π , que especifica cómo se eligen las acciones a partir de los estados u observaciones disponibles. La *señal de recompensa* (*reward signal*) define el objetivo inmediato del problema, mientras que la *función de valor* estima la conveniencia de estados o acciones en términos de recompensa futura esperada. Sutton y Barto también distinguen el *modelo* del entorno, entendido como una descripción que permite predecir sus transiciones y recompensas; cuando dicho modelo no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1 El Proceso de Decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el Proceso de Decisión de Markov (*Markov Decision Process*, MDP). En un MDP finito, el agente y el entorno interactúan en una secuencia de pasos temporales discretos $t = 0, 1, 2, \dots$. En cada instante t , el agente recibe una representación del estado del entorno $S_t \in \mathcal{S}$ y selecciona una acción $A_t \in \mathcal{A}(S_t)$. Como consecuencia de esa acción, el entorno emite una recompensa

numérica $R_{t+1} \in \mathcal{R}$ y transita a un nuevo estado S_{t+1} [SB18]. Este ciclo se representa en la Figura 2.7.

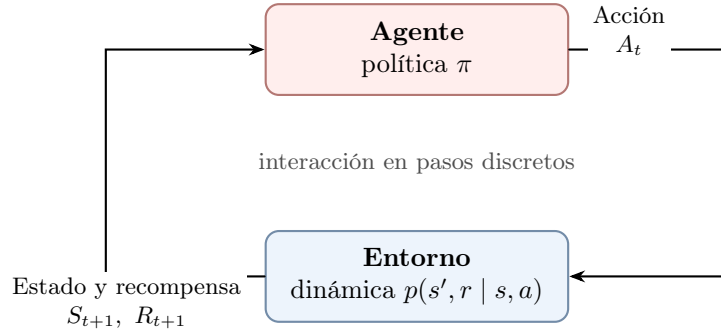


Figura 2.7: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r | s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s :

$$p(s', r | s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\} \quad (2.5)$$

donde $S_t \in \mathcal{S}$ representa el estado en el instante t , $A_t \in \mathcal{A}$ es la acción seleccionada, $R_{t+1} \in \mathbb{R}$ es la recompensa escalar recibida y $S_{t+1} \in \mathcal{S}$ es el nuevo estado resultante.

La propiedad de Markov establece que, dado el estado y la acción actuales, la distribución del próximo estado y recompensa no depende explícitamente de la historia completa de estados y acciones anteriores. En otras palabras, el estado debe contener toda la información relevante del pasado para predecir la evolución futura del proceso [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un Proceso de Decisión de Markov Parcialmente Observable (POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t , que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.6)$$

donde G_t representa la suma acumulada descontada de recompensas futuras a partir del instante t , y γ es el factor de descuento. Un valor de γ cercano a cero prioriza recompensas inmediatas, mientras que un valor cercano a uno otorga un peso significativo a recompensas futuras de largo plazo [ACS24].

2.2.2 Métodos Basados en Valor y Basados en Política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

En los *métodos basados en valor*, la estrategia consiste en estimar qué tan conveniente es encontrarse en un estado o ejecutar una acción determinada. Esto se formaliza mediante la

función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$. Ambas funciones satisfacen ecuaciones recursivas conocidas como *ecuaciones de Bellman*, que expresan la relación entre el valor de una situación actual y el valor esperado de sus posibles sucesores [SB18].

Cuando la dinámica del entorno no se conoce completamente, estas funciones pueden estimarse a partir de la experiencia. Los métodos de Diferencia Temporal (*Temporal-Difference*, TD) combinan el muestreo de transiciones reales con actualizaciones basadas en estimaciones previas, mecanismo conocido como *bootstrapping*. Un ejemplo central es *Q-learning*, que actualiza la función de acción-valor hacia una estimación de la función óptima $q_*(s, a)$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad (2.7)$$

donde $Q(S_t, A_t)$ es la estimación del valor de la acción A_t en el estado S_t , $\alpha \in (0, 1]$ representa la tasa de aprendizaje (*learning rate*), R_{t+1} es la recompensa observada, γ es el factor de descuento, y $\max_a Q(S_{t+1}, a)$ estima el valor máximo alcanzable en el siguiente estado S_{t+1} .

Por el contrario, los *métodos basados en política* parametrizan directamente la política $\pi(a \mid s, \theta) = \Pr\{A_t = a \mid S_t = s, \theta_t = \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables (pesos neuronales). En lugar de derivar la acción exclusivamente de una función de valor, ajustan los parámetros θ para maximizar una medida de rendimiento escalar $J(\theta)$. Para ello aplican ascenso de gradiente sobre el rendimiento esperado:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \quad (2.8)$$

donde $\nabla J(\theta_t)$ representa el gradiente de la función de rendimiento respecto a los parámetros de la política. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona la base teórica para estimar este gradiente sin derivar explícitamente la distribución de estados inducida por la política [SB18]. En la práctica, muchos algoritmos modernos combinan ambas ideas mediante arquitecturas actor-crítico: un actor representa la política y un crítico estima valores para guiar sus actualizaciones.

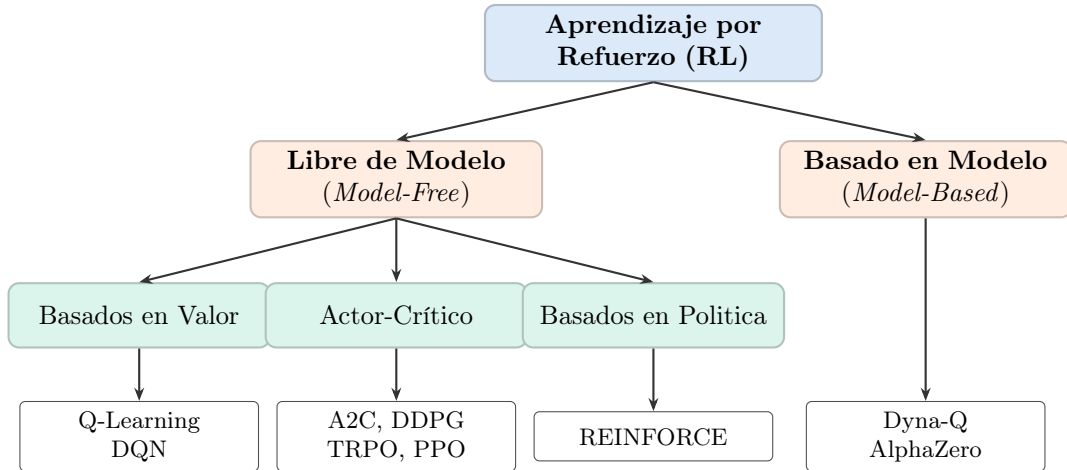


Figura 2.8: Taxonomía de algoritmos de Aprendizaje por Refuerzo, enfocada en métodos *Model-Free* (adaptada de OpenAI *Spinning Up* [Ach18]).

2.2.3 Aprendizaje Profundo y Redes Neuronales

La Unidad Neuronal y Redes *Feedforward*

En problemas de control complejos, los métodos tabulares se vuelven impracticables debido a las restricciones de memoria y cómputo que impone la maldición de la dimensionalidad [SB18].

La aproximación de funciones soluciona esta limitación al representar funciones mediante una forma parametrizada, permitiendo que el aprendizaje en un estado se generalice a otros estados con características similares. El aprendizaje profundo (*Deep Learning*) emplea redes neuronales artificiales como aproximadores de funciones continuas y no lineales [ACS24].

El componente básico de estas arquitecturas es la unidad neuronal individual. Computacionalmente, una neurona en una capa k ejecuta dos operaciones secuenciales sobre el vector de características de entrada $\mathbf{x}$. Primero, calcula una combinación lineal utilizando un vector de pesos $\mathbf{w}$ y un sesgo (*bias*) escalar b . Luego, el resultado se somete a una función de activación no lineal g_k :

$$f_{k,u}(\mathbf{x}; \theta_k^u) = g_k(\mathbf{w}^\top \mathbf{x} + b) \quad (2.9)$$

donde $\mathbf{x} \in \mathbb{R}^d$ es el vector de entrada, $\mathbf{w} \in \mathbb{R}^d$ representa los pesos asociativos, $b \in \mathbb{R}$ es el sesgo, y g_k es la función de activación diferenciable. La inclusión de una función de activación no lineal es estrictamente necesaria; componer múltiples capas sin funciones no lineales resultaría analíticamente en una simple transformación lineal, anulando la capacidad de representación de la red [ACS24]. Entre las opciones no lineales más comunes destacan ReLU (Unidad Lineal Rectificada, $g(z) = \max(0, z)$), *leaky* ReLU ($g(z) = \max(\alpha z, z)$ con $\alpha \ll 1$) y Tanh (tangente hiperbólica, comprimiendo las salidas en $[-1, 1]$).

Las redes neuronales de propagación hacia adelante (*feedforward* o perceptrones multicapa) estructuran estas unidades en secuencias de múltiples capas: una capa de entrada, capas intermedias u ocultas, y una capa de salida. La computación de una capa entera se puede vectorizar:

$$\mathbf{h}_k = g_k(\mathbf{W}_k^\top \mathbf{x}_{k-1} + \mathbf{b}_k) \quad (2.10)$$

donde $\mathbf{W}_k$ es la matriz de pesos de la capa k , $\mathbf{x}_{k-1}$ es la salida activada de la capa previa y $\mathbf{b}_k$ es el vector de sesgos. El Teorema de Aproximación Universal establece que una red con una única capa oculta suficientemente ancha puede aproximar cualquier función continua; sin embargo, arquitecturas más profundas logran una mejor generalización y una extracción jerárquica de características progresivamente más abstractas [ACS24, SB18].

El Proceso de Optimización

Dado que las redes neuronales carecen de una solución de forma cerrada, sus parámetros θ se ajustan utilizando métodos de optimización iterativa. El proceso se fundamenta en tres componentes técnicos [ACS24]:

- **Función de Pérdida (*Loss Function*):** Un objetivo matemático diferenciable $\mathcal{L}(\theta)$ que cuantifica la discrepancia entre las predicciones de la red y los valores objetivo. Frecuentemente se emplea el Error Cuadrático Medio (MSE).
- **Descenso de Gradiente:** El optimizador actualiza los parámetros moviéndose en la dirección del gradiente negativo $-\nabla_{\theta} \mathcal{L}(\theta)$, ponderado por una tasa de aprendizaje α . En la práctica se utiliza el Descenso de Gradiente por Mini-lotes (*Mini-batch Gradient Descent*), que muestrea un subconjunto aleatorio de datos para ofrecer un equilibrio óptimo entre la estabilidad del gradiente y el costo computacional. Optimizadores modernos como Adam adaptan dinámicamente las tasas de aprendizaje para acelerar la convergencia.
- **Retropropagación (*Backpropagation*):** Para que el descenso de gradiente sea posible en arquitecturas profundas, es necesario computar las derivadas parciales de la función de pérdida respecto a cada parámetro. El algoritmo de *backpropagation* logra esto aplicando eficientemente la regla de la cadena de derivadas, propagando los gradientes desde la capa de salida hacia las capas de entrada.

2.2.4 Aprendizaje por Refuerzo Profundo (DRL)

El Aprendizaje por Refuerzo Profundo (DRL) combina los algoritmos de RL con la capacidad de generalización del aprendizaje profundo. Esta integración es fundamental cuando los espacios de estado son continuos o de alta dimensionalidad, donde es estadísticamente improbable que un agente visite el mismo estado múltiples veces. Sin embargo, el DRL exige aproximadores robustos capaces de procesar flujos de datos correlacionados temporalmente y de rastrear objetivos no estacionarios [ACS24].

El Problema de la Tríada Mortal (*Deadly Triad*)

La aplicación directa de redes neuronales al aprendizaje por refuerzo introduce vulnerabilidades analíticas. Sutton y Barto [SB18] establecen que la inestabilidad de los parámetros y la divergencia de los valores están garantizadas matemáticamente si un algoritmo combina simultáneamente tres elementos fundamentales:

- **Aproximación de Funciones:** Uso de aproximadores potentes y escalables, como las redes neuronales, que extienden el aprendizaje generalizando a través del espacio de estados.
- **Bootstrapping:** La práctica de actualizar estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (como en los métodos TD), en lugar de esperar la consecución de retornos reales completos.
- **Entrenamiento *Off-Policy*:** Optimizar la función de valor utilizando una distribución de datos o transiciones generadas por una política de comportamiento distinta a la política objetivo que se desea evaluar o mejorar.

El análisis de la Tríada Mortal argumenta que es fácticamente imposible prescindir de cualquiera de estos tres vértices en sistemas de control escalables [SB18]. Renunciar a la aproximación de funciones restringiría el control a problemas tabulares diminutos. Descartar el *bootstrapping* mitigaría la inestabilidad, pero paralizaría la eficiencia de datos y la viabilidad computacional al requerir almacenar secuencias episódicas enteras. Finalmente, sacrificar el aprendizaje *off-policy* anularía la capacidad del agente de aprender de forma concurrente múltiples modelos predictivos a partir de un único flujo de experiencia. Por consiguiente, resulta imperativo tolerar la Tríada Mortal e incorporar mitigaciones algorítmicas estructurales.

Deep Q-Networks (DQN)

En los métodos basados en valor, DRL sustituye la tabla clásica por una red neuronal $Q(s, a; \theta)$. En espacios de acción discretos, el algoritmo *Deep Q-Networks* (DQN) procesa el estado s como vector de entrada y computa simultáneamente las estimaciones de valor para todas las acciones en su capa de salida [ACS24, MKS⁺15].

La inestabilidad descrita por la Tríada Mortal se manifiesta en DQN como el “problema del blanco móvil”: al generalizar, una actualización que incrementa el valor de un estado modifica de forma inadvertida los valores de otros estados. Si además las secuencias de entrenamiento presentan una alta correlación temporal, la red sufre de olvido catastrófico, sobreajustándose a la experiencia más reciente. Para lograr convergencia, DQN introduce dos innovaciones estructurales imperativas [ACS24]:

- **Redes Objetivo (*Target Networks*):** Se emplea una red neuronal secundaria separada cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular los valores objetivo en el error TD, amortiguando las oscilaciones y estabilizando el blanco de aprendizaje.

- **Búferes de Repetición (*Replay Buffers*):** Se almacenan las transiciones históricas del agente (s, a, r, s') . El entrenamiento se realiza extrayendo mini-lotes aleatorios de esta memoria, lo cual destruye la correlación temporal intrínseca de los procesos de decisión de Markov y estabiliza la varianza del gradiente.

La función de pérdida minimizada por DQN es:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_j, s_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_j + \gamma \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor Objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción Actual}} \right)^2 \right] \quad (2.11)$$

donde $\mathcal{D}$ es el búfer de repetición de experiencias, (s_j, a_j, r_j, s_{j+1}) representa una transición muestreada, θ indica los parámetros de la red Q actual y θ^- representa los parámetros congelados de la red objetivo.

Algorithm 1 Algoritmo *Deep Q-Networks* (DQN) con *Replay Buffer*

- 1: Inicializar la memoria de repetición $\mathcal{D}$ con capacidad N
 - 2: Inicializar la red de acción-valor Q con pesos aleatorios θ
 - 3: Inicializar la red objetivo $\hat{Q}$ con pesos $\theta^- = \theta$
 - 4: **for** episodio = 1 hasta M **do**
 - 5: Inicializar secuencia y obtener estado inicial s_1
 - 6: **for** $t = 1$ hasta T **do**
 - 7: Seleccionar una acción a_t mediante política ϵ -greedy a partir de $Q(s_t, a; \theta)$
 - 8: Ejecutar acción a_t , observar recompensa r_t y siguiente estado s_{t+1}
 - 9: Almacenar transición (s_t, a_t, r_t, s_{t+1}) en $\mathcal{D}$
 - 10: Muestrear un minilote aleatorio de transiciones (s_j, a_j, r_j, s_{j+1}) desde $\mathcal{D}$
 - 11: Calcular objetivo $y_j = \begin{cases} r_j & \text{si } s_{j+1} \text{ es terminal} \\ r_j + \gamma \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-) & \text{en otro caso} \end{cases}$
 - 12: Realizar un paso de descenso de gradiente respecto a la pérdida $(y_j - Q(s_j, a_j; \theta))^2$ para ajustar θ
 - 13: Cada C pasos, sincronizar la red objetivo: $\theta^- \leftarrow \theta$
 - 14: **end for**
 - 15: **end for**
-

Aproximación de Políticas, Función de Ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s; \theta)$ [ACS24].

Para cuantificar el desempeño marginal de una acción tomada en un estado específico respecto a la expectativa promedio de la política, se define la **Función de Ventaja** (*Advantage Function*) $A^\pi(s, a)$ [SB18]:

$$A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s) \quad (2.12)$$

donde $Q^\pi(s, a)$ representa el retorno esperado al tomar la acción a en el estado s bajo la política π , y $V^\pi(s)$ es el valor promedio de estado. Conceptualmente, la ventaja mide cuánto mejor ($A^\pi(s, a) > 0$) o peor ($A^\pi(s, a) < 0$) resulta la acción a en comparación con el desempeño medio del agente en el estado s .

En arquitecturas actor-crítico, la ventaja se estima habitualmente mediante la Estimación Generalizada de la Ventaja (*Generalized Advantage Estimation*, GAE) desarrollada por Schulman

et al. [SML⁺16]:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} \doteq \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (2.13)$$

donde $\delta_t^V = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ representa el error de diferencia temporal (TD) estimado por la red del crítico V , $\gamma \in [0, 1)$ es el factor de descuento y $\lambda \in [0, 1]$ es el hiperparámetro de suavizado GAE que equilibra el sesgo y la varianza del estimador.

En lugar de minimizar un error cuadrático de predicción, la optimización paramétrica del actor se rige por el Teorema del Gradiente de Política (*Policy Gradient Theorem*), que establece la actualización de los parámetros θ ascendiendo por el gradiente del rendimiento esperado [SB18, SML⁺16]:

$$\nabla_{\theta} J(\theta) = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \hat{A}_t \right] \quad (2.14)$$

donde $\nabla_{\theta} J(\theta)$ representa el gradiente del desempeño esperado, $\pi_{\theta}(A_t | S_t)$ es la probabilidad otorgada a la acción ejecutada A_t en el estado S_t , y $\hat{A}_t$ es la ventaja estimada. Esta formulación incrementa la probabilidad de elegir acciones con ventaja positiva ($\hat{A}_t > 0$) y reduce la probabilidad de aquellas con ventaja negativa.

Proximal Policy Optimization (PPO) y Mejora Monótona

Una dificultad conocida de los métodos de gradiente de política tradicionales (como REINFORCE o A2C) es su sensibilidad a la tasa de aprendizaje: actualizaciones de parámetros con pasos demasiado grandes pueden modificar la distribución de la política de forma abrupta, provocando un colapso catastrófico del desempeño del cual el agente raramente se recupera.

Para resolver esta vulnerabilidad, Kakade y Langford [KL02] y Schulman et al. (TRPO [SLA⁺15]) formalizaron el principio de **Mejora Monótona de la Política** (*Monotonic Policy Improvement*). Este principio garantiza matemáticamente que en cada iteración de optimización k , el retorno esperado del agente es monótonamente no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \quad (2.15)$$

donde $J(\pi_k)$ representa el desempeño esperado de la política en la iteración k , y $J(\pi_{k+1})$ es el desempeño esperado tras actualizar sus parámetros. Mientras que TRPO logra esta garantía imponiendo una restricción severa sobre la divergencia Kullback-Leibler (KL) mediante optimización cuadrática de segundo orden, *Proximal Policy Optimization* (PPO) [SWD⁺17] aproxima este comportamiento mediante un objetivo sustituto recortado (*clipped surrogate objective*) computable mediante descenso de gradiente de primer orden.

Dada la razón de probabilidad entre la política actual y la anterior:

$$r_t(\theta) \doteq \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2.16)$$

donde $\pi_{\theta}(a_t | s_t)$ es la probabilidad de la acción a_t bajo los nuevos parámetros θ , y $\pi_{\theta_{\text{old}}}(a_t | s_t)$ es la probabilidad registrada durante la recolección del lote, PPO maximiza el siguiente objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.17)$$

donde $\epsilon > 0$ es un hiperparámetro de recorte (típicamente $\epsilon \approx 0.1$ – 0.2), y $\text{clip}(r, 1 - \epsilon, 1 + \epsilon)$ acota la razón r al intervalo $[1 - \epsilon, 1 + \epsilon]$. El operador $\min$ toma el valor más pesimista entre la ventaja ponderada simple y la ventaja recortada, eliminando el incentivo a realizar actualizaciones que modifiquen excesivamente la política si la ventaja es positiva, y acotando la penalización si la ventaja es negativa.

En la práctica, la función objetivo completa de PPO combina el término de política, la pérdida de ajuste del crítico y un término de entropía para favorecer la exploración:

$$\mathcal{L}^{\text{PPO}}(\theta, \phi) = \hat{\mathbb{E}}_t \left[L^{\text{CLIP}}(\theta) - c_1 \left(V_\phi(s_t) - V_t^{\text{target}} \right)^2 + c_2 S[\pi_\theta](s_t) \right] \quad (2.18)$$

donde ϕ representa los parámetros de la red del crítico V_ϕ , V_t^{target} son los retornos observados objetivo, $S[\pi_\theta]$ denota la entropía de la distribución de la política, y $c_1, c_2 > 0$ son coeficientes de ponderación escalar.

PPO es un algoritmo *on-policy* caracterizado por su estabilidad y simplicidad. En esta tesina, su importancia no se limita a su rol individual, sino que constituye la base algorítmica sobre la cual se estructuran tanto el *baseline* independiente (IPPO) como las extensiones cooperativas multi-agente (MAPPO y HAPPO).

Algorithm 2 Algoritmo *Proximal Policy Optimization* (PPO) - Variante *Actor-Critic*

- 1: Inicializar parámetros de política (actor) θ y función de valor (crítico) ϕ
 - 2: **for** iteración = 1, 2, ... **do**
 - 3: Recolectar un conjunto de trayectorias $\mathcal{D}_k = \{\tau_i\}$ ejecutando la política actual π_θ en el entorno
 - 4: Calcular los valores del crítico $V_\phi(s)$ y las estimaciones de ventaja $\hat{A}_t^{\text{GAE}}$
 - 5: Optimizar el objetivo $L^{\text{CLIP}}(\theta)$ respecto a θ durante K épocas (*epochs*) sobre minilotes de $\mathcal{D}_k$
 - 6: Optimizar la función de pérdida del crítico $L^{\text{VF}}(\phi) = (V_\phi(s_t) - \hat{V}_t)^2$ respecto a ϕ
 - 7: **end for**
-

2.3 Aprendizaje por Refuerzo Multi-Agente (MARL)

El aprendizaje por refuerzo multi-agente (*Multi-Agent Reinforcement Learning*, MARL) generaliza los principios del RL hacia sistemas donde múltiples entidades aprenden e interactúan simultáneamente en un entorno compartido [ACS24]. A diferencia del caso individual, las acciones de cada agente modifican el estado global del sistema, lo que induce una dinámica no estacionaria desde la perspectiva local de cualquier participante. En el contexto de sistemas dinámicos cooperativos, múltiples controladores coordinan sus decisiones para optimizar un objetivo común.

2.3.1 Fundamentos Teóricos: Juegos Estocásticos, Equilibrio de Nash y Dec-POMDP

La formalización matemática de los entornos multi-agente requiere extender el MDP individual incorporando nociones de la Teoría de Juegos. El modelo matemático fundamental para representar decisiones secuenciales con múltiples participantes es el Juego Estocástico o Juego Markoviano (*Stochastic Game* o *Markov Game*), introducido por Shapley [Sha53] y adaptado al aprendizaje por refuerzo por Albrecht et al. [ACS24].

Un Juego Estocástico para n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, \{R_i\}_{i=1}^n, \gamma \rangle \quad (2.19)$$

donde $\mathcal{N} = \{1, \dots, n\}$ representa el conjunto de agentes, $\mathcal{S}$ es el espacio de estados globales, $\mathcal{A}_i$ es el espacio de acciones del agente i , $P(s' | s, \mathbf{a})$ representa la función de transición de estados condicionada a la acción conjunta $\mathbf{a} = (a_1, \dots, a_n) \in \mathcal{A} \equiv \mathcal{A}_1 \times \dots \times \mathcal{A}_n$, y $R_i(s, \mathbf{a})$ representa la función de recompensa asignada al agente i . En un entorno estrictamente cooperativo, todos

los agentes comparten exactamente la misma señal de recompensa ($R_1 = R_2 = \dots = R_n = R$), de modo que el objetivo de cada agente es maximizar el retorno global esperado del sistema.

En el marco de la Teoría de Juegos, la solución teórica a la que aspiran los algoritmos de MARL es el **Equilibrio de Nash** [ACS24]. Una política conjunta $\pi^* = (\pi_1^*, \dots, \pi_n^*)$ constituye un Equilibrio de Nash si ningún agente $i \in \mathcal{N}$ puede incrementar unilateralmente su retorno esperado desviándose de su política π_i^* , asumiendo que los demás agentes mantienen sus políticas fijas en π_{-i}^* :

$$J(\pi_i, \pi_{-i}^*) \leq J(\pi_i^*, \pi_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \quad (2.20)$$

donde $\pi^* = (\pi_1^*, \dots, \pi_n^*)$ es la combinación de políticas individuales en equilibrio, π_i representa cualquier política alternativa ejecutable por el agente i , Π_i denota el espacio de políticas válidas del agente i , y $\pi_{-i}^* = (\pi_1^*, \dots, \pi_{i-1}^*, \pi_{i+1}^*, \dots, \pi_n^*)$ es el vector de políticas fijas de todos los demás agentes excepto el agente i . En términos sencillos, un Equilibrio de Nash representa un punto de convergencia donde ninguna entidad tiene un incentivo individual para alterar unilateralmente su estrategia.

En escenarios reales de control distribuido, ningún agente dispone de visibilidad completa del estado global del sistema. Cada participante recopila información únicamente de sus sensores locales. Por consiguiente, los entornos cooperativos multi-agente con observabilidad parcial se formalizan matemáticamente mediante el marco de los Procesos de Decisión de Markov Parcialmente Observables Descentralizados (Dec-POMDP) [ACS24, AOS20].

Un Dec-POMDP para n agentes se define como una tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, R, \{\Omega_i\}_{i=1}^n, O, \gamma \rangle \quad (2.21)$$

donde Ω_i denota el conjunto de observaciones individuales del agente i , regidas por la función de observación probabilística $O(o_1, \dots, o_n \mid s', \mathbf{a}) = \Pr\{O_1 = o_1, \dots, O_n = o_n \mid S' = s', \mathbf{A} = \mathbf{a}\}$. En este esquema, cada agente i basa sus decisiones únicamente en su historial u observación local $o_i \in \Omega_i$, con el propósito de aprender una política descentralizada $\pi_i(a_i \mid o_i)$ que, al combinarse con las políticas de los demás agentes en la política conjunta $\pi = (\pi_1, \dots, \pi_n)$, maximice el retorno acumulado conjunto del sistema [AOS20].

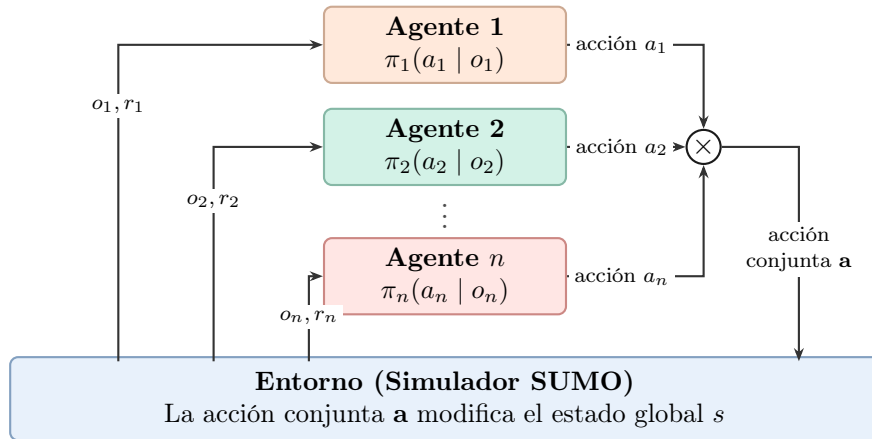


Figura 2.9: Interacción Agente-Entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

2.3.2 Desafíos Fundamentales en MARL

La aplicación práctica de métodos de aprendizaje por refuerzo en escenarios multi-agente enfrenta obstáculos analíticos y computacionales significativos que no existen en el caso individual [ACS24]. A medida que aumenta el número de agentes, el espacio de acciones conjunto $\mathcal{A}$ crece

de forma exponencial con el número de agentes ($|\mathcal{A}| = \prod_{i=1}^n |\mathcal{A}_i|$), lo que se conoce como la maldición de la dimensionalidad multi-agente. Sin embargo, los desafíos teóricos más severos radican en la dinámica propia de la interacción:

- **No Estacionariedad:** Desde la perspectiva de un agente individual i , las políticas de los demás agentes actúan como parte de la dinámica de transición del entorno. Como todos los agentes aprenden y adaptan sus pesos neuronales de manera concurrente, la función de transición efectiva percibida localmente por el agente i :

$$\mathcal{P}_i(s' | s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' | s, a_i, \mathbf{a}_{-i}) \pi_{-i}(\mathbf{a}_{-i} | s) \quad (2.22)$$

cambia de forma continua a lo largo del entrenamiento. Aquí $\mathbf{a}_{-i}$ representa las acciones conjuntas del resto de los agentes y π_{-i} sus respectivas políticas. Esta co-adaptación rompe la Propiedad de Markov base desde la perspectiva local e invalida las garantías de convergencia de los algoritmos de agente único [ACS24, FNF⁺17].

- **Observabilidad Parcial:** Dado el modelo Dec-POMDP inherente, la falta de telemetría global restringe la visión de cada agente a variables puramente locales. Esto introduce estocasticidad e incertidumbre en la varianza de los gradientes evaluados, dificultando la estimación precisa de las funciones de valor.
- **Asignación de Crédito Multi-Agente (*Credit Assignment*):** En juegos de recompensa común, resulta complejo discernir analíticamente la contribución o el demérito marginal que la acción de un agente aislado produjo sobre el desempeño sistémico global. Una decisión local subóptima podría recibir un refuerzo positivo si el resto del sistema operó de manera sumamente eficiente, sesgando las actualizaciones de la política [ACS24].

2.3.3 Paradigmas de Entrenamiento y Ejecución

Para abordar estas patologías en escenarios Dec-POMDP, la literatura clasifica los algoritmos en función de cómo fluye la información durante el proceso de optimización. Se distinguen fundamentalmente dos enfoques: el Aprendizaje Independiente (*Independent Learning*, IL) y el Entrenamiento Centralizado con Ejecución Descentralizada (*Centralized Training with Decentralized Execution*, CTDE) [ZLYZ23, ACS24].

En el primero, los agentes se entrenan localmente utilizando métodos de agente único, tratando empíricamente al resto de los agentes como parte de la dinámica del entorno. En el segundo, durante la fase de entrenamiento simulado (*offline*), los algoritmos explotan información global privilegiada (historiales conjuntos o concatenación de observaciones) para guiar y estabilizar la optimización de las funciones de valor. Una vez finalizado el entrenamiento, el componente centralizado se descarta y los actores ejecutan políticas condicionadas estrictamente a observaciones locales en tiempo real (*online*).

2.3.4 Multi-Agent Deep Reinforcement Learning (MADRL) y Algoritmos Baseline

La integración de los formalismos Dec-POMDP con la capacidad de aproximación generalizada del aprendizaje profundo ha dado origen al campo del *Multi-Agent Deep Reinforcement Learning* (MADRL) [LWT⁺17, ACS24]. Esta área agrupa los enfoques prevalentes para el control adaptativo de sistemas complejos a gran escala.

En esta investigación, para evaluar rigurosamente el desempeño de las arquitecturas de entrenamiento centralizado (MAPPO y HAPPO), resulta metodológicamente indispensable establecer **líneas de base (*baselines*) de aprendizaje independiente (IL)**. En los algoritmos independientes, cada coordinador actúa como un agente aislado que ignora la presencia explícita de los demás participantes.

Aprendizaje Independiente (IL): IDQN e IPPO como Algoritmos Baseline

- **Independent DQN (IDQN) como Baseline Basado en Valor:** Representa la extensión multi-agente más directa de Q-Learning profundo [TMK⁺17]. Su principal vulnerabilidad metodológica radica en que, al ser un algoritmo *off-policy*, depende críticamente del remuestreo de transiciones desde un *Replay Buffer*. En un entorno MARL no estacionario, esas memorias corresponden a configuraciones históricas donde los demás agentes operaban bajo políticas pasadas. Entrenar sobre estos datos obsoletos agrava la inestabilidad descrita por la Tríada Mortal, provocando la divergencia del valor y el colapso del aprendizaje [FNF⁺17]. En la evaluación experimental de este trabajo, IDQN sirve como *baseline* representativo de los métodos basados en valor independientes.
- **Independent PPO (IPPO) como Baseline Basado en Política:** Asigna una instancia del algoritmo PPO a cada agente. A diferencia de IDQN, la literatura ha demostrado empíricamente que IPPO exhibe una mayor robustez técnica frente a la no estacionariedad [dWGM⁺20]. Esta resiliencia se fundamenta en su naturaleza *on-policy*: al descartar memorias pasadas y extraer gradientes únicamente de datos muestreados en proximidad temporal directa con la política conjunta actual, asegura que las actualizaciones respondan a la dinámica vigente del sistema. Adicionalmente, el recorte paramétrico (clip) previene saltos perjudiciales en el espacio de la política. IPPO constituye el *baseline* independiente principal contra el cual se contrastan los algoritmos coordinados.

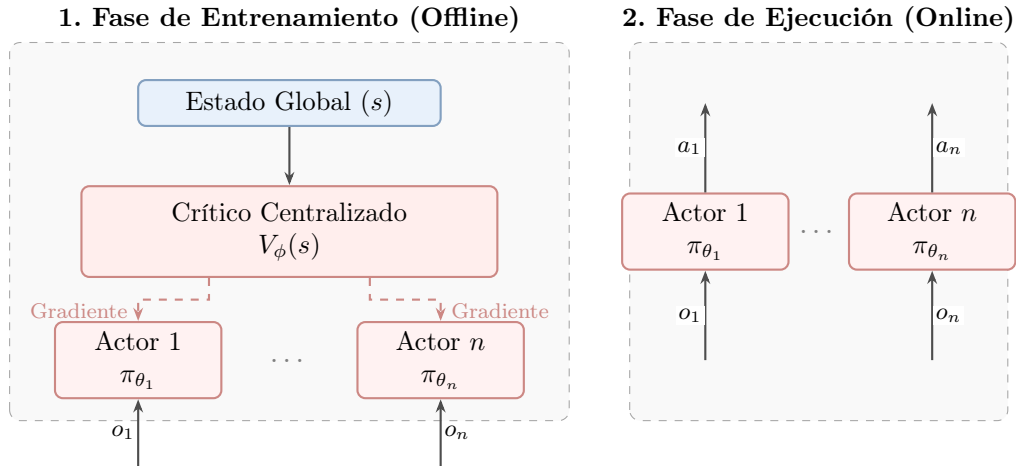


Figura 2.10: Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). El Crítico utiliza información global privilegiada para guiar las actualizaciones de los Actores locales durante la simulación.

2.3.5 Multi-Agent Proximal Policy Optimization (MAPPO)

Para abordar la asignación de crédito y la no estacionariedad sin perder la capacidad de ejecución descentralizada, Yu et al. [YVV⁺22] introdujeron *Multi-Agent Proximal Policy Optimization* (MAPPO), adaptando el algoritmo PPO de agente único al entorno multi-agente cooperativo bajo el paradigma CTDE (véase la Figura 2.10).

En MAPPO, la estructura distingue entre actores descentralizados y un crítico centralizado:

- **Actores localizados ($\pi_{\theta_i}(a_i | o_i)$):** Cada agente i mantiene una red de política paramétrica que procesa únicamente su observación local o_i y emite una distribución sobre las acciones locales a_i . En entornos con agentes homogéneos, es habitual utilizar compartición de parámetros (*Parameter Sharing*), entrenando una única red de actor compartida por todos los agentes pero evaluada en sus observaciones individuales.

- **Crítico centralizado** ($V_\phi(s)$): Durante el entrenamiento, una red de función de valor global $V_\phi(s)$ recibe el estado global s del entorno (o la concatenación del vector de observaciones de todos los agentes $s = (o_1, \dots, o_n)$). El crítico estima el valor esperado del retorno conjunto G_t y computa la ventaja conjunta $\hat{A}_t$ utilizando GAE [YVV⁺22].

El objetivo de optimización del actor en MAPPO extiende la pérdida recortada de PPO a la acción conjunta:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t), \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \quad (2.23)$$

donde $r_t^i(\theta) = \frac{\pi_\theta(a_t^i | o_t^i)}{\pi_{\theta_{\text{old}}}(a_t^i | o_t^i)}$ es la razón de probabilidad individual del agente i , n es el número total de agentes, $\hat{A}_t(s_t, \mathbf{a}_t)$ es la ventaja conjunta evaluada por el crítico centralizado, y ϵ es el hiperparámetro de recorte.

Por su parte, el crítico centralizado se actualiza minimizando el error cuadrático medio respecto al retorno objetivo descontado normalizado:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] \quad (2.24)$$

donde ϕ son los parámetros del crítico centralizado y R_t^{target} representa el retorno empírico objetivo.

Factores Críticos de Diseño y Optimización en MAPPO

A pesar de su simplicidad conceptual, Yu et al. [YVV⁺22] demostraron empíricamente que la efectividad de MAPPO depende fuertemente de cinco decisiones de diseño e hiperparámetros específicos:

1. **Normalización del Valor (Estadísticas Móviles y Algoritmo PopArt)**: En juegos cooperativos complejos, las variaciones continuas en la política conjunta provocan fluctuaciones drásticas en la escala del retorno acumulado R_t^{target} , lo que desestabiliza la regresión de la red del crítico. MAPPO resuelve esta inestabilidad estandarizando dinámicamente los objetivos de valor. Esto se logra mediante el cálculo de la media y la desviación estándar móviles de los retornos observados (*Running Statistics*), o bien mediante la aplicación del algoritmo PopArt (*Preserving Outputs Precisely under Adaptive Rescaling Targets* [HSE⁺19]), el cual reescala los objetivos ajustando adaptativamente los pesos de la capa final del crítico. Al computar el estimador de ventaja GAE, las salidas del crítico se desnormalizan para conservar la escala física de los rendimientos al guiar la actualización del actor.
2. **Representación del Estado Global en el Crítico**: Yu et al. [YVV⁺22] evaluaron distintas opciones para estructurar la entrada s del crítico centralizado:
 - Concatenación de Observaciones Locales (*Concatenated Local, CL*): forma el estado uniendo las observaciones locales de todos los agentes $s = (o_1, \dots, o_n)$.
 - Estado Global del Entorno (*Environment Provided, EP*): utiliza un vector de estado global provisto directamente por el simulador.
 - Estado Específico por Agente (*Agent-Specific State, AS*): combina el estado del entorno con la información local del agente i ($AS_i = EP \cup o_i$).
 - Filtrado de Características Redundantes (*Feature Pruning, FP*): remueve variables locales duplicadas o de baja varianza en el vector AS_i .

El estudio concluye que utilizar la combinación de características específicas del agente con filtrado de redundancias ($AS + FP$) equilibra la precisión predictiva del crítico sin inflar la dimensionalidad de entrada.

3. **Reutilización Restringida de Datos (Épocas y Minilotes):** Mientras que PPO de agente único suele reutilizar la experiencia recolectada durante 20 o más épocas divididas en múltiples minilotes pequeños, en MARL la no estacionariedad torna peligrosa la reutilización excesiva. MAPPO restringe la optimización a un máximo de $K \in [5, 15]$ épocas por actualización y recomienda emplear 1 o 2 subdivisiones de minilotes para evitar un desfase grave respecto a la política de recolección.
4. **Rango Estricto de Recorte (Clipping ϵ):** Para limitar la deriva destructiva de la política conjunta inducida por el aprendizaje concurrente, MAPPO requiere mantener un valor de recorte $\epsilon \leq 0.2$ (típicamente $\epsilon \in [0.05, 0.1]$). Este rango acota las desviaciones en la distribución de acciones conjuntas, asegurando transiciones estables hacia patrones de coordinación.
5. **Tamaño de Lote (Batch Size):** Existe un umbral crítico de muestras *on-policy* por actualización por debajo del cual la varianza del estimador de ventaja impide descubrir la coordinación entre nodos. Superado este tamaño de lote mínimo, el rendimiento se estabiliza.

Algorithm 3 Algoritmo *Multi-Agent Proximal Policy Optimization* (MAPPO)

- 1: Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
 - 2: Inicializar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
 - 3: **for** iteración = 1, 2, ... **do**
 - 4: Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta π_θ
 - 5: Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
 - 6: Calcular los retornos objetivos R_t^{target} normalizados para el crítico
 - 7: Almacenar las transiciones $(s_t, o_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
 - 8: **for** época = 1 hasta K **do**
 - 9: Muestrear minilotes de datos desde $\mathcal{D}$
 - 10: Actualizar la política minimizando la pérdida de recorte PPO conjunta:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t, \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$
 - 11: Actualizar el crítico ϕ minimizando el error cuadrático medio del valor:

$$\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$$
 - 12: **end for**
 - 13: Vaciar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
 - 14: **end for**
-

2.3.6 Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

En sistemas de control complejos, los agentes presentan con frecuencia asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En estos escenarios, la práctica habitual de compartir parámetros entre los agentes (*Parameter Sharing*) impone una homogeneización forzada que restringe la especialización táctica necesaria de cada nodo. Por otro lado, entrenar políticas independientes sin compartir parámetros (como en IPPO) carece de garantías teóricas de convergencia y suele conducir a oscilaciones destructivas debido al aprendizaje simultáneo no coordinado [KCW⁺21].

Para solucionar esta limitación, Kuba et al. [KCW⁺21] y Zhong et al. [ZKF⁺24] formularon el paradigma de Aprendizaje Multi-Agente Heterogéneo (*Heterogeneous-Agent Reinforcement Learning*, HARL) y desarrollaron el algoritmo *Heterogeneous-Agent Proximal Policy Optimization* (HAPPO). HAPPO preserva la heterogeneidad de los actores permitiendo que cada agente i posea su propia arquitectura de red neuronal y parámetros θ^i adaptados a su espacio de acciones local $\mathcal{A}_i$, manteniendo las garantías teóricas de mejora monótona del rendimiento conjunto y convergencia a un Equilibrio de Nash [KCW⁺21].

Compartición de Experiencia vs. Compartición de Parámetros

Una distinción central entre MAPPO y HAPPO radica en la gestión de los modelos y la experiencia:

- **Compartición de Parámetros (*Parameter Sharing*):** Utilizado habitualmente por MAPPO, impone que todos los agentes compartan los mismos pesos neuronales $\theta_1 = \dots = \theta_n = \theta$. Esto exige que los espacios de observación y acción de todos los agentes sean homogéneos.
- **Compartición de Experiencia (*Experience Sharing*):** Adoptado por HAPPO, permite que cada agente mantenga una red de actor independiente $\pi_{\theta^i}(a_i | o_i)$ con topología y pesos propios. No obstante, en lugar de entrenar n críticos individuales, todos los agentes comparten un único crítico centralizado $V_\phi(s)$ y reutilizan el mismo lote de trayectorias colectivas. Esto logra una elevada eficiencia de datos sin forzar simetría en los actores.

Lema de Descomposición de la Ventaja Multi-Agente

El soporte matemático de HAPPO se fundamenta en el Lema de Descomposición de la Ventaja Multi-Agente propuesto por Kuba et al. [KCW⁺21].

En cualquier juego markoviano cooperativo, para una política conjunta π y una permutación ordenada arbitraria de los agentes $i_{1:n} = (i_1, i_2, \dots, i_n) \in \text{Sym}(n)$, la ventaja de una acción conjunta $\mathbf{a} = (a_{i_1}, \dots, a_{i_n})$ en el estado s se descompone exactamente como la suma de las ventajas marginales de cada agente tomadas secuencialmente, condicionadas por las acciones de los agentes que lo preceden en el ordenamiento:

$$A^\pi(s, \mathbf{a}) = \sum_{m=1}^n A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \quad (2.25)$$

donde $A^\pi(s, \mathbf{a})$ representa la ventaja conjunta global, y la ventaja marginal del agente i_m se define formalmente como:

$$A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}}) \quad (2.26)$$

donde $Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ representa el valor esperado de la acción conjunta considerando las acciones fijadas por los primeros m agentes en la secuencia, y $Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}})$ es el valor marginalizado sobre los agentes restantes.

En esta formulación, las acciones de los agentes que aún no han actualizado su política en la secuencia ($\mathbf{a}_{i_{m+1:n}}$) se marginalizan por esperanza matemática bajo la política conjunta vigente. La gran relevancia teórica de este lema es que se cumple para cualquier juego cooperativo de recompensa común sin requerir ningún supuesto de aditividad o monotonía sobre la función de valor, a diferencia de los esquemas de descomposición de valor como VDN [SLG⁺17] o QMIX [RSS⁺18].

Esquema de Actualización Secuencial y Factor de Razón Compuesto M

Para explotar la descomposición de la ventaja sin incurrir en la inestabilidad del entrenamiento simultáneo, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración de optimización k .

En cada época de entrenamiento k , se genera una permutación aleatoria de agentes $i_{1:n} \in \text{Sym}(n)$. El sorteo aleatorio evita que un agente monopolice la prioridad de actualización, garantizando la convergencia hacia un Equilibrio de Nash [ZKF⁺24, WZK⁺23].

Cuando le corresponde el turno de optimización al agente i_m , los agentes anteriores en la secuencia $(i_{1:m-1})$ ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores $(i_{m+1:n})$ conservan sus parámetros originales $\theta_k^{i_j}$. Para guiar la actualización del agente i_m respetando la modificación sufrida por la distribución conjunta debido a los agentes precedentes, Kuba et al. [KCW⁺21] demostraron que no es necesario entrenar un crítico separado por cada subgrupo, sino que la ventaja marginal ajustada se puede calcular multiplicando la ventaja conjunta $A^{\pi^k}(s, \mathbf{a})$ por la productoria de los ratios de probabilidad de los agentes que ya se actualizaron.

Se define así el factor de razón de política compuesto $M_{i_{1:m}}(s, \mathbf{a})$:

$$M_{i_{1:m}}(s, \mathbf{a}) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}}^{i_j}(a^{i_j} | o^{i_j})}{\pi_{\theta_k}^{i_j}(a^{i_j} | o^{i_j})} \right) A^{\pi^k}(s, \mathbf{a}) \quad (2.27)$$

donde $\pi_{\theta_{k+1}}^{i_j}$ es la política recién optimizada del agente precedido i_j , $\pi_{\theta_k}^{i_j}$ es su política previa, y $A^{\pi^k}(s, \mathbf{a})$ es la ventaja conjunta de la red. Para el primer agente de la secuencia ($m = 1$), la productoria es vacía y se define como $M_{i_1}(s, \mathbf{a}) \equiv A^{\pi^k}(s, \mathbf{a})$.

El factor $M_{i_{1:m}}(s, \mathbf{a})$ actúa como un modificador de importancia que ajusta la señal de ventaja para el agente i_m , absorbiendo analíticamente el desfase en la distribución de la acción conjunta provocado por las actualizaciones precedentes sin necesidad de realizar nuevas simulaciones en el entorno.

De este modo, para optimizar la política del agente i_m , HAPPO maximiza el objetivo sustituto recortado de PPO ponderado por la razón compuesta $M_{i_{1:m}}$:

$$L^{\text{HAPPO}}(\theta^{i_m}) = \mathbb{E}_t \left[\min \left(r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \text{clip} \left(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon, 1 + \epsilon \right) M_{i_{1:m}}(s_t, \mathbf{a}_t) \right) \right] \quad (2.28)$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente i_m .

Una vez realizada la actualización de descenso de gradiente sobre θ^{i_m} , es indispensable recalcular la razón de probabilidad post-optimización $\rho_{\text{post}}^{i_m} = \frac{\pi_{\theta_{k+1}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ y actualizar recursivamente el factor M para los agentes restantes:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t) \quad (2.29)$$

Garantía de Mejora Monótona

La combinación de la descomposición exacta de la ventaja con el recorte de la razón de política compuesta en HAPPO permite demostrar el siguiente resultado analítico central [KCW⁺21, ZKF⁺24]:

En cada iteración de optimización k , la actualización secuencial de los actores garantiza que el retorno esperado conjunto del sistema es monótonamente no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \quad (2.30)$$

donde $J(\pi_k)$ representa el rendimiento esperado de la política conjunta en el paso k . Esta propiedad de **mejora monótona** elimina las oscilaciones destructivas que caracterizan al aprendizaje independiente (IPPO/IDQN) y proporciona una base teórica sólida para garantizar la convergencia a un Equilibrio de Nash local en sistemas multi-agente complejos.

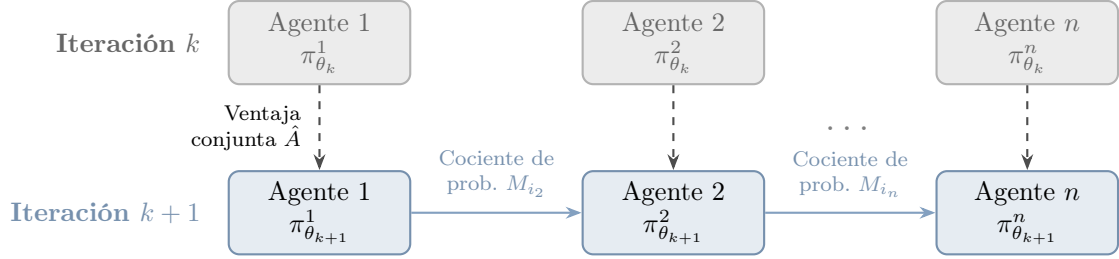


Figura 2.11: Esquema de actualización secuencial de HAPPO. A diferencia de PPO Independiente, la actualización de cada agente está matemáticamente condicionada por el cociente de probabilidad compuesto M_{i_m} , que propaga los cambios efectuados por los agentes que lo precedieron en la secuencia.

Algorithm 4 Algoritmo *Heterogeneous-Agent Proximal Policy Optimization* (HAPPO)

- 1: Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
 - 2: **for** iteración = 1, 2, ... **do**
 - 3: Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta π_θ
 - 4: Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
 - 5: Sorteo aleatorio de una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
 - 6: Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \dots, n\}$
 - 7: **for** $m = 1 \dots n$ **do**
 - 8: Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
 - 9: Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando la pérdida recortada HAPPO:

$$L^{\text{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \left[\min \left(r_t^{i_m}(\theta^{i_m}) M_{i_m}(s_t, \mathbf{a}_t), \text{clip} \left(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon, 1 + \epsilon \right) M_{i_m}(s_t, \mathbf{a}_t) \right) \right]$$
 - 10: Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k}^{i_m}(a_t^{i_m} | o_t^{i_m})}$
 - 11: **for** $j = m + 1 \dots n$ **do**
 - 12: Actualizar el factor modificador para agentes subsecuentes:

$$M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$$
 - 13: **end for**
 - 14: **end for**
 - 15: Actualizar el crítico centralizado ϕ mediante descenso de gradiente:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$$
 - 16: **end for**
-

Para finalizar el estudio comparativo de los algoritmos basados en Optimización de Política Próxima en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Cuadro 2.3: Comparación sintética de familias algorítmicas PPO en entornos multi-agente cooperativos.

Eje de Análisis	PPO Agente Único	IPPO (Independiente)	MAPPO (CTDE Homogéneo)	HAPPO (HARL Heterogéneo)
Paradigma	Agente Único	DTE (Descentralizado)	CTDE (Centralizado/Desc.)	CTDE / HARL
Arquitectura de Actores	Único $\pi_\theta(a s)$	n Actores independientes o compartidos	Compartición de Parámetros (π_θ único)	n Actores heterogéneos (π_{θ^i})
Arquitectura del Crítico	Único $V_\phi(s)$	n Críticos locales $V_{\phi_i}(o_i)$	Un Crítico Centralizado $V_\phi(s)$	Un Crítico Centralizado $V_\phi(s)$
Mecanismo de Optimización	Simultáneo / Directo	Simultáneo independiente	Simultáneo paralelo	Secuencial con Permutación $i_{1:n}$
Señal de Ventaja	Ventaja local $\hat{A}(s, a)$	Ventaja local $\hat{A}_i(o_i, a_i)$	Ventaja conjunta $\hat{A}(s, \mathbf{a})$	Factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora Monótona	Garantizada (TRPO/PPO)	Sin garantía (no estacionario)	Sin garantía monótona estricta	Garantizada monótona ($J_{k+1} \geq J_k$)
Soporte para Heterogeneidad	No aplica	Limitado e inestable	Limitado (exige Parameter Sharing)	Nativo y matemáticamente exacto

2.4 Control de Tráfico Urbano mediante MARL

Una vez establecidos los marcos teóricos de la Ingeniería de Tránsito y del Aprendizaje por Refuerzo Multi-Agente, esta sección formaliza el mapeo directo entre los conceptos físicos del dominio del tráfico y la tupla de decisión secuencial Dec-POMDP, definiendo la arquitectura de interacción entre el simulador y los controladores inteligentes.

2.4.1 Formalización del Control Semafórico Coordinado como Dec-POMDP

En el contexto del control adaptativo de red, cada intersección semaforizada se modela como un agente inteligente $i \in \mathcal{N} = \{1, \dots, n\}$. La interacción entre la red vial simulada en SUMO y el sistema de control se formaliza mediante los elementos de la tupla Dec-POMDP:

- **Entorno (*environment*) e Interfaz TraCI:** El simulador microscópico SUMO actúa como el entorno dinámico. A través de la interfaz TraCI (*Traffic Control Interface*) y la librería de abstracción SUMO-RL [Ale], se establece un bucle de control discreto paso a paso donde el sistema lee la telemetría del tráfico y envía comandos de fase a los semáforos.
- **Espacio de observaciones ($o_i \in \Omega_i$):** Vector de características locales registrado en los accesos de la intersección i en el instante t . Incluye la longitud de cola por carril (q_l), la densidad u ocupación (k_l), la velocidad media espacial (v_l), el número de vehículos detenidos y el indicador de la fase semafórica activa actualmente.
- **Espacio de acciones ($a_i \in \mathcal{A}_i$):** Selección discreta de la siguiente fase semafórica a activar o mantener. Cada decisión se ejecuta durante un tiempo de verde fijo g (e.g., 5 segundos). Si el agente decide cambiar de fase, el entorno intercala de forma automática los intervalos de seguridad física ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) antes de iniciar la nueva fase.
- **Función de recompensa (R_i):** En el marco abstracto del Dec-POMDP, la recompensa actúa como una señal escalar de realimentación diseñada para incentivar la fluidez vehicular penalizando las manifestaciones físicas de la congestión (colas, demoras y detenciones).

La especificación matemática exacta y las distintas funciones de recompensa evaluadas y comparadas en este trabajo se formulan formalmente en el Capítulo 3 (Metodología).

- **Coordinación y prevención de congestión (*Spillback*):** Las decisiones operativas de un semáforo afectan de forma directa a las intersecciones adyacentes. Una fase verde prolongada en un acceso principal descarga un pelotón de vehículos que avanza hacia el nodo contiguo; si ese nodo no ajusta oportunamente su temporización, la cola resultante puede bloquear la intersección aguas arriba (*spillback*). Mediante algoritmos CTDE (MAPPO) y HARL (HAPPO), los agentes aprenden a coordinar secuencias de verde continuas (olas verdes adaptativas), maximizando la capacidad de descarga global.

2.4.2 Funciones de Recompensa en Control Semafórico

En la formalización de un Dec-POMDP para el control semafórico adaptativo, la función de recompensa R_i constituye la señal de realimentación cuantitativa mediante la cual el sistema guía el aprendizaje de los agentes [ne25]. En este trabajo, el marco computacional **mar1TLC** formaliza e implementa un conjunto jerárquico de funciones de recompensa orientadas a resolver el compromiso entre la optimización local y la coordinación sistémica:

- **Formulaciones Diferenciales y Absolutas:** Las recompensas absolutas, como la demora negativa ($R = -W_{\text{acc}}$) o la longitud de cola negativa ($R = -N_{\text{queue}}$), penalizan permanentemente el estado actual del tráfico. Sin embargo, en simulación microscópica con flujos dinámicos, esta penalización absoluta acumula deriva de varianza a lo largo del episodio. Las formulaciones diferenciales computan la mejora discreta respecto al paso temporal anterior:

$$R_{\text{diff_halted}} = N_{\text{halted}}(t-1) - N_{\text{halted}}(t) \quad (2.31)$$

$$R_{\text{diff_waiting}} = W_{\text{acc}}(t-1) - W_{\text{acc}}(t) \quad (2.32)$$

donde un valor positivo indica una reducción efectiva en el número de vehículos detenidos o en la demora acumulada, proporcionando un gradiente de progreso local paso a paso que estabiliza la asignación de crédito temporal [DK11].

- **Presión de Tráfico (*PressureReward*):** Basada en la teoría de flujo de saturación y control de presión máxima (*Max-Pressure*) [WCZ⁺19], la presión P_i en una intersección i evalúa la diferencia entre los vehículos detenidos en los carriles entrantes (L_{in}) y los vehículos en los carriles salientes (L_{out}):

$$P_i = \left| \sum_{l \in L_{\text{in}}} N_{\text{halted},l} - \sum_{m \in L_{\text{out}}} N_{\text{halted},m} \right| \quad (2.33)$$

Definiendo $R_{\text{pressure}} = -P_i$, la maximización del incentivo equivale a igualar las tasas de descarga y prevenir la acumulación de presión aguas abajo.

- **Normalización por Volumen Vehicular:** Para evitar que intersecciones con mayor volumen dominen desproporcionadamente la señal de aprendizaje en redes heterogéneas, se aplica la normalización por la cantidad de vehículos activos:

$$R_{\text{diff_waiting_norm}} = \frac{W_{\text{acc}}(t-1) - W_{\text{acc}}(t)}{\max(N_{\text{veh}}(t), 1)} \quad (2.34)$$

- **Mezcla Cooperativa Vecinal (λ -mix):** Para mitigar el comportamiento egoísta o miope de agentes independientes, la función **NeighborMixReward** combina de manera convexa la

señal local L_i con la media de las recompensas observadas en las intersecciones vecinas N_i :

$$R_i = (1 - \lambda)L_i + \lambda \left(\frac{1}{|N_{ei}(i)|} \sum_{j \in N_{ei}(i)} L_j \right) \quad (2.35)$$

donde $\lambda = \frac{w}{1+w} \in [0, 1)$ parametriza la fracción de cooperación vecinal (siendo $w \geq 0$ el peso de mezcla histórica). Esta señal compartida promueve la coordinación espacial y favorece la aparición espontánea de olas verdes entre nodos adyacentes.

- **Penalización por Desbordamiento de Cola (*Spillback Penalty*):** Cuando la densidad en los carriles salientes supera el umbral crítico de capacidad ($k \geq 95\%$), la señal `SpillbackPenaltyReward` inyecta un castigo escalar severo. Esto desincentiva acciones que saturan los accesos contiguos, bloqueando físicamente el cruce aguas arriba (*spillback*).

2.5 Frameworks Tecnológicos de Escalado y Optimización

La evaluación experimental de algoritmos de Aprendizaje por Refuerzo Multi-Agente Profundo en escenarios de simulación microscópica exige infraestructuras de cómputo capaces de escalar la recolección de experiencia y optimizar iterativamente espacios de hiperparámetros de alta dimensionalidad.

2.5.1 Ecosistema Ray: Ray Core, Ray Tune y RLlib

Ray es un framework de computación distribuida de código abierto diseñado para escalar aplicaciones de inteligencia artificial y aprendizaje por refuerzo [Any25]. En esta investigación, la pila tecnológica se apoya en tres módulos fundamentales del ecosistema Ray:

- **Ray Core:** Infraestructura distribuida subyacente basada en el modelo de actores y tareas asíncronas. Permite gestionar de manera paralela múltiples procesos de simulación en SUMO interactuando a través de TraCI en entornos multinúcleo o distribuidores de cómputo HPC [Any25].
- **Ray Tune:** Plataforma distribuida para la selección de modelos y optimización de hiperparámetros [LLN⁺18b]. Ray Tune coordina la ejecución paralela de pruebas (*trials*), gestiona dinámicamente los recursos de hardware (CPU/GPU) por experimento y registra métricas de telemetría en tiempo real.
- **RLlib:** Librería de referencia para aprendizaje por refuerzo distribuido masivo [LLN⁺18a]. Ofrece abstracciones nativas para entornos multi-agente bajo el paradigma CTDE (MAP-PO y HAPPO), soportando compartición de parámetros (*Parameter Sharing*), vectorización paralela de entornos TraCI y recolección eficiente de trayectorias *on-policy*.

2.5.2 Framework Optuna y Pipeline de Optimización Multietapa

Para sintonizar de forma rigurosa los hiperparámetros algorítmicos (como la tasa de aprendizaje α , el rango de recorte PPO ϵ , el número de épocas K y la entropía), se emplea el framework de optimización automática Optuna [ASY⁺19, Opt25].

Optimización Bayesiana TPE y Poda Asíncrona ASHA

La exploración estocástica del espacio de búsqueda en Optuna se estructura mediante dos componentes algorítmicos principales:

- **Muestreo Bayesiano TPE (*Tree-structured Parzen Estimator*):** A diferencia de la búsqueda aleatoria o en grilla, TPE construye estimadores de densidad condicional no paramétricos $l(x)$ y $g(x)$ para modelar las distribuciones de hiperparámetros asociadas a los desempeños superiores e inferiores observados, respectivamente [ASY⁺19]. El algoritmo muestrea iterativamente las regiones que maximizan la razón de verosimilitud (*Likelihood Ratio*), concentrando la exploración en las cuencas de convergencia más prometedoras.
- **Poda Asíncrona ASHA (*Asynchronous Successive Halving Algorithm*):** Dado el costo computacional de ejecutar simulaciones completas en SUMO, ASHA monitorea asincrónicamente las curvas de aprendizaje intermedias y cancela tempranamente (*pruning*) las pruebas que se ubican en los percentiles inferiores de desempeño [LJR⁺20]. Se define un período de gracia (*grace period*) adecuado para preservar pruebas MARL cuya convergencia coordinada suele ser lenta en las fases iniciales.

Pipeline Multietapa y Métrica de Retorno Robusto al Riesgo (CVaR)

Con el propósito de mitigar la maldición de la dimensionalidad sin introducir sesgo humano temprano, la optimización se organiza en un **Pipeline Multietapa**:

1. **Fase 1 (Exploración Estocástica de Amplio Espectro - *Broad Search*):** Explora el espacio global mediante distribuciones no restringidas en Optuna (loguniform, uniform extensos), identificando las cuencas de atracción del algoritmo en distintos escenarios viales.
2. **Fase 2 (Refinamiento Bayesiano Guiado por Evidencia - *Refined Search*):** Bajo el principio de la **Navaja de Ockham Bayesiana**, refina las distribuciones probabilísticas alrededor de los valles óptimos de la Fase 1. Si un parámetro convergió contra el límite del rango inicial (censura de frontera), el método expande formalmente sus fronteras para verificar la existencia del óptimo global.

En entornos MARL, los algoritmos sufren con frecuencia de **colapso de política** (*Policy Collapse*), una degradación repentina de las políticas cooperativas causada por la alta varianza del aprendizaje concurrente [YVV⁺22]. Para forzar a Optuna a seleccionar únicamente configuraciones estables y seguras, la función objetivo abandona el promedio aritmético simple en favor de una métrica de **Retorno Robusto al Riesgo** inspirada en el Valor en Riesgo Condicional (CVaR):

$$\text{Robust_Return} = \mathbb{E}[\mathcal{R}] - \alpha \cdot (\mathbb{E}[\mathcal{R}] - \min(\mathcal{R})) \quad (2.36)$$

donde $\mathbb{E}[\mathcal{R}]$ representa la recompensa promedio esperada del lote, $\min(\mathcal{R})$ es la peor recompensa episódica registrada en el lote, y $\alpha \in [0, 1]$ es el factor de aversión al riesgo (típicamente $\alpha = 0.5$). Penalizar la varianza inferior descarta configuraciones inestables y garantiza la mejora monótona del desempeño conjunto en el sistema de control.

Metodología y Diseño Experimental

En este capítulo se detalla el marco metodológico de la investigación, la operacionalización de las variables, el diseño de los escenarios de simulación, la arquitectura de los componentes computacionales y los protocolos experimentales adoptados para evaluar las hipótesis planteadas. El estudio se encuadra dentro de un enfoque ****cuantitativo, aplicado y experimental puro****, donde la simulación microscópica de tránsito actúa como el laboratorio controlado para evaluar la capacidad de aprendizaje, coordinación y robustez de las distintas arquitecturas de Aprendizaje por Refuerzo Multi-Agente (MARL).

3.1 Enfoque y Diseño de Investigación

De acuerdo con la tipología metodológica de investigación en ciencias computacionales [ACS24], este trabajo adopta un ****diseño experimental cuantitativo de medidas repetidas****. Se manipulan sistemáticamente variables independientes (familias algorítmicas, grado de cooperación espacial y escenarios de demanda vial) en entornos computacionales controlados para medir sus efectos sobre variables dependientes primarias (demora acumulada W_{net} , tasa de completitud CR y emisiones contaminantes de CO_2).

La utilización de la simulación microscópica (mediante el simulador SUMO) como plataforma experimental primaria responde a tres razones científicas:

1. **Seguridad y Viabilidad Operacional:** Probar algoritmos estocásticos de control semafórico en tiempo real sobre una red urbana real implica un riesgo inaceptable para la seguridad vial y la fluidez del tránsito.
2. **Aislamiento de Variables Extrañas:** La simulación permite neutralizar perturbaciones estocásticas no controladas (condiciones meteorológicas, incidentes viales o fluctuaciones aleatorias fuera de modelo), garantizando la validez interna del estudio.
3. **Replicabilidad Exacta:** Permite someter a todas las familias de control (algoritmos independientes, coordinados y de tiempo fijo) exactamente a la misma secuencia determinista de demanda mediante la fijación rigurosa de semillas aleatorias.

La Figura 3.1 ilustra la secuencia metodológica estructurada en siete fases interconectadas que rige esta investigación.

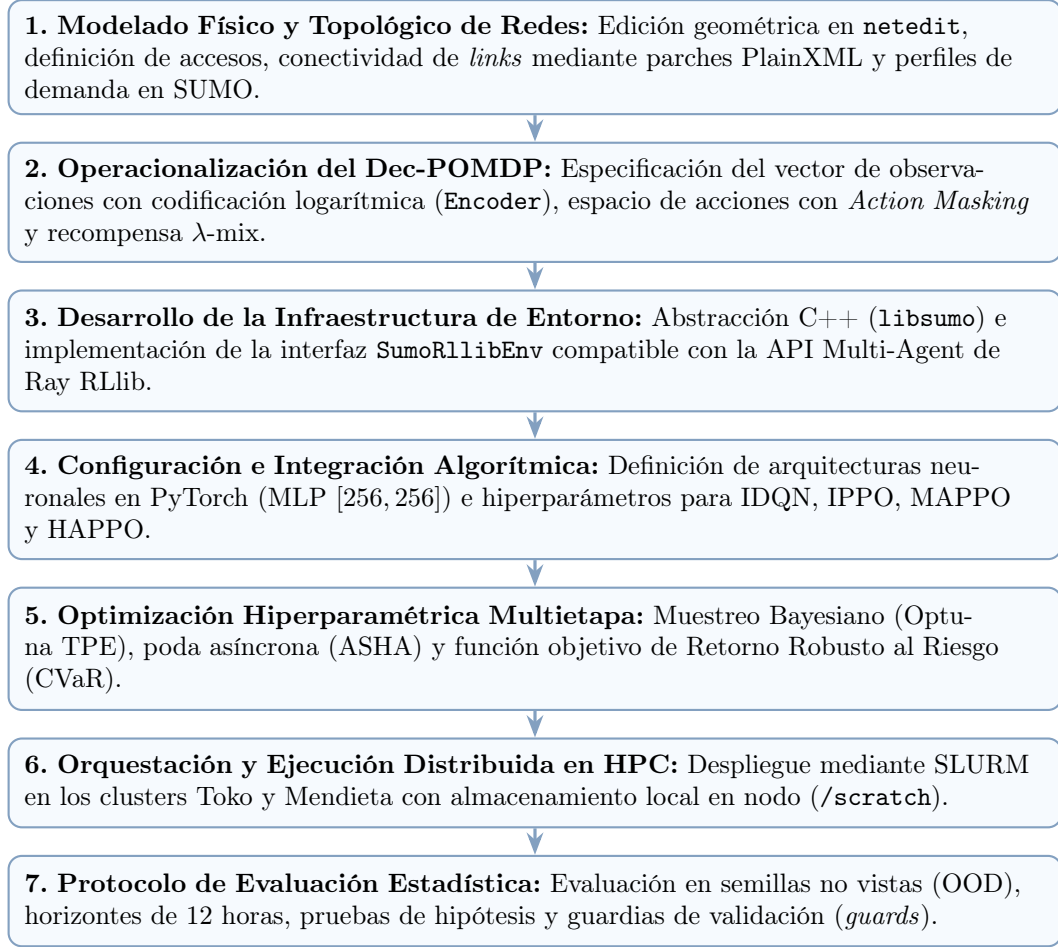


Figura 3.1: Flujo metodológico secuencial de la investigación experimental.

3.2 Especificación del Stack Tecnológico

Para garantizar la reproducibilidad científica, la Tabla 3.1 detalla el ecosistema de software y hardware sobre el cual se desarrolló y ejecutó el marco computacional *mar1TLC*.

Cuadro 3.1: Especificación del stack tecnológico utilizado en la investigación.

Componente	Herramienta / Versión	Función Metodológica
Lenguaje de Programación	Python 3.12	Núcleo del sistema y orquestación.
Simulador de Tránsito	SUMO 1.20+ / TraCI	Simulación microscópica y telemetría de red.
Binding de Simulación	libsumo (C++)	Ejecución de simulación de alta velocidad en memoria.
Framework MARL	Ray RLLib 2.x	Abstracción de entornos multi-agente y algoritmos CTDE.
Aprendizaje Profundo	PyTorch 2.x	Arquitectura de redes neuronales (Actor-Crítico MLP).
Optimización Automática	Optuna 3.x	Muestreo Bayesiano TPE y poda asíncrona ASHA.
Infraestructura HPC	Clusters Toko y Mendieta	Cómputo distribuido multinúcleo gestionado vía SLURM.

3.3 Diseño de Escenarios de Tránsito en SUMO

La evaluación de los algoritmos se realiza sobre cuatro escenarios de red en SUMO, construidos mediante la herramienta gráfica `netedit` y scripts en Python, cubriendo un espectro representativo de complejidades topológicas y dinámicas de flujo.

3.3.1 Escenarios Sintéticos: 3×1 , 3×3 y $4 \times 4_{-H}$

Todas las redes sintéticas comparten reglas operacionales estandarizadas:

- **Límite de Velocidad:** 13.89 m/s (50 km/h) en todas las arterias urbanas.
- **Mezcla Vehicular Heterogénea:** Calibrada para representar un parque automotor urbano realista mediante tres clases de vehículos en el archivo `urban_vehicle_mix.add.xml`:
 1. **SLOWVEHTYPE** (25%): Conductores cautelosos o vehículos pesados ($a = 1.9 \text{ m/s}^2$, $b = 4.0 \text{ m/s}^2$, $v_{\max} = 30 \text{ km/h}$).
 2. **MEDVEHTYPE** (60%): Vehículo de pasajeros promedio ($a = 2.6 \text{ m/s}^2$, $b = 4.5 \text{ m/s}^2$, $v_{\max} = 50 \text{ km/h}$).
 3. **FASTVEHTYPE** (15%): Conductores agresivos ($a = 3.1 \text{ m/s}^2$, $b = 5.0 \text{ m/s}^2$, $v_{\max} = 70 \text{ km/h}$ acotado a 50 km/h por el límite del carril).

Las tres topologías sintéticas cumplen funciones analíticas complementarias:

- **Corredor Arterial 3×1 :** Tres nodos dispuestas en línea recta con flujo bidireccional dominante (3.260 veh/h). Permite evaluar la capacidad de los agentes para sincronizar olas verdes sin interferencia de lazos cerrados.
- **Malla Simétrica 3×3 :** Nueve intersecciones organizadas en cuadrícula cerrada (7.087 veh/h). Al contener lazos de dependencia circular, constituye el escenario crítico para evaluar la resiliencia de los algoritmos frente al colapso por desbordamiento de cola (*gridlock*).
- **Cuadrícula Heterogénea $4 \times 4_{-H}$:** Dieciséis intersecciones asimétricas (7.202 veh/h) con arterias principales de mayor capacidad que actúan como vías de escape para drenar la congestión.

3.3.2 Escenario Urbano Real: Gran Mendoza (Corredor Vicente Zapata)

Para evaluar la transferibilidad de las políticas entrenadas hacia un entorno urbano real, se construyó el escenario del Gran Mendoza a partir de datos geoespaciales de OpenStreetMap y calibración física.

- **Procesamiento Topológico:** La geometría de la red fue procesada mediante la herramienta `netconvert` y parches PlainXML (`.con.xml`) para preservar la conectividad de giros y prohibiciones de maniobra observadas en terreno.
- **Calibración de Flujos Empíricos:** Se utilizaron los datos de aforo vehicular continuo registrados por la Cámara 169 en el corredor Vicente Zapata.
- **Matriz Origen-Destino Híbrida:** La demanda vehicular combina un componente fijo espacialmente explícito (60% de la demanda distribuida en 476 pares OD calibrados) con un remanente estocástico (40% restante) generado por `randomRoutesTaz` con la semilla congelada 104773.
- **Calibración Dinámica:** El escenario simula un horizonte operacional continuo de 12 horas (06 : 00–18 : 00), alcanzando un estadístico de calidad de flujo $GEH < 5$ en más del 85% de las mediciones de volumen en los enlaces comparables.

—

3.4 Arquitectura de Integración Entorno-Agentes

El marco computacional `mar1TLC` integra el simulador SUMO con la librería Ray RLlib a través de la interfaz `SumoRllibEnv`.

3.4.1 Ciclo Episódico e Interfaz de Simulación

Para acelerar la recolección de experiencias, las simulaciones se ejecutan mediante la abstracción C++ `libsumo` (configurada mediante la variable de entorno `LIBSUMO_AS_TRACI=1`).

Cada episodio inyecta un período de **calentamiento físico** (*warmup*) de **600 segundos** bajo control semafórico de tiempo fijo. Al finalizar el *warmup*, el estado microscópico completo de la simulación se congela en memoria. Los llamados a la función `reset()` restauran directamente dicho punto de control (*checkpoint*), garantizando que el entrenamiento de los agentes MARL inicie siempre en un régimen de tránsito saturado realista.

Las decisiones se ejecutan en pasos discretos de $\Delta t = 5.0$ s. Si un agente decide cambiar la fase activa, el entorno intercala automáticamente la secuencia de seguridad física obligatoria:

$$t_{\text{transición}} = t_{\text{amarillo}}(3 \text{ s}) + t_{\text{rojo_todo}}(2 \text{ s}) \quad (3.1)$$

—

3.5 Instanciación del Modelo Dec-POMDP

La interacción distribuida entre las intersecciones semaforizadas se formaliza mediante la tupla Dec-POMDP instanciada en la librería `mar1TLC`.

3.5.1 Espacio de Observaciones y Codificación Logarítmica

La observación $o_i^{(t)}$ de la intersección i en el paso t combina la longitud de cola por carril entrante (q_l), la velocidad media (v_l), el estado de la fase activa y el tiempo de espera acumulado cuantizado (w_l).

Para evitar que valores extremos de tiempo de espera distorsionen la escala del vector de entrada en las redes neuronales, se aplica la clase `Encoder` (`src/marlTLC/utils/encoder.py`).

En esta formulación:

- **Número de Intervalos (I):** Corresponde al hiperparámetro `encoder_intervals`, el cual define la resolución del espacio discreto de observación y es optimizado automáticamente mediante exploración Bayesiana en Optuna.
- **Cota de Saturación del Carril (M):** Corresponde al parámetro `max_lane_value`, el cual se determina empíricamente antes del entrenamiento mediante el script de calibración `scripts/calibrate_encoder_m.py`. El procedimiento ejecuta simulaciones preliminares del escenario objetivo y calcula el percentil 99 de los tiempos de espera acumulados observados en todos los carriles controlados. Los valores calibrados consolidados (almacenados en `.encoder_M.json`) son $M = 1184$ s para el escenario 3×1 , $M = 1136$ s para 3×3 , $M = 712$ s para $4 \times 4_H$ y $M = 1494$ s para Gran Mendoza.

La función cuantiza la variable continua x mediante la regla logarítmica:

$$e(x) = \text{ceil} \left(F \cdot \log_2 \left(\frac{x}{M \cdot L} + 1 \right) + L^2 \cdot x \right) \quad (3.2)$$

donde $L = \frac{I-1}{M}$ y $F = (1-L) \frac{I-1}{\log_2(1/L+1)}$, mapeando el valor continuo x al rango discreto $[0, I-1]$.

3.5.2 Espacio de Acciones y Enmascaramiento Dinámico

El espacio de acciones $\mathcal{A}_i = \{0, 1, \dots, K_i - 1\}$ representa el conjunto de fases verdes seleccionables por la intersección i . En cruces con geometrías asimétricas, la capa `ActionMaskingTorchRLModule` aplica un enmascaramiento sobre los *logits* $z_i(a)$ emitida por el actor:

$$\text{logit}_i(a) = \begin{cases} z_i(a) & \text{si la fase } a \text{ es geométricamente válida} \\ -10^9 & \text{si la fase } a \text{ es inválida} \end{cases} \quad (3.3)$$

3.5.3 Función de Recompensa Compuesta y Coordinación Vecinal

La función de recompensa implementada en este trabajo resuelve el compromiso entre el rendimiento local y la cooperación distribuida sin recurrir a términos heurísticos adicionales.

1. **Recompensa Local Diferencial Normalizada (L_i):** Mide la variación en el tiempo de espera acumulado por vehículo entre pasos temporales consecutivos:

$$L_i^{(t)} = \frac{W_{\text{acc},i}(t-1) - W_{\text{acc},i}(t)}{\max(N_{\text{veh},i}(t), 1)} \quad (3.4)$$

2. **Recompensa Compartida Vecinal (N_i):** Corresponde al promedio de las recompensas locales obtenidas por los nodos pertenecientes al vecindario topológico directo $N(i)$:

$$N_i^{(t)} = \frac{1}{|N(i)|} \sum_{j \in N(i)} L_j^{(t)} \quad (3.5)$$

3. **Mezcla Convexa λ -mix (NeighborMixReward)**: Combina ambas señales mediante el parámetro de cooperación $\lambda \in [0, 1]$:

$$R_i^{(t)} = (1 - \lambda)L_i^{(t)} + \lambda N_i^{(t)} \quad (3.6)$$

donde $\lambda = \frac{w}{1+w}$, siendo $w \geq 0$ el peso de ponderación vecinal.

3.6 Implementación Algorítmica e Hiperparámetros

Los algoritmos de aprendizaje independiente (IDQN e IPPO) y de entrenamiento centralizado (MAPPO y HAPPO) se implementaron en PyTorch e integraron en la API de Ray RLlib mediante el módulo `algo_factory.py`. La Tabla 3.2 resume la parametrización utilizada.

Cuadro 3.2: Especificación de hiperparámetros algorítmicos finales.

Hiperparámetro	Valor Seleccionado	Ámbito de Aplicación
Arquitectura Neuronal	MLP [256, 256]	Redes de Actor y Crítico.
Tasa de Aprendizaje (α)	3×10^{-4}	Optimizador Adam.
Factor de Descuento (γ)	0.99	Retorno descontado episódico.
Parámetro GAE (λ_{GAE})	0.95	Estimación de ventaja (PPO/MAPPO/HAPPO).
Rango de Recorte PPO (ϵ)	0.19	Pérdida recortada L^{CLIP} .
Tamaño de Lote de Entrenamiento	4000 muestras	Buffer por iteración.
Tamaño de Minilote	128 muestras	Subdivisión para gradiente.
Épocas por Iteración (K)	10	Reutilización de datos <i>on-policy</i> .

3.7 Diseño de Experimentos de Búsqueda Hiperparamétrica

La exploración de hiperparámetros se ejecutó en dos fases utilizando la integración entre Optuna, Ray Tune y el algoritmo de poda ASHA:

1. **Fase 1 (Exploración Bayesiana)**: Búsqueda amplia con muestreo TPE sobre 40 configuraciones por escenario.
2. **Fase 2 (Refinamiento Bayesiano)**: Expansión de los rangos de búsqueda tras detectar censura de frontera en el peso de mezcla vecinal ($w \in [1.0, 5.0]$).

Para evitar políticas inestables, la función objetivo optimiza el **Retorno Robusto al Riesgo** (inspirado en CVaR):

$$\text{Robust_Return} = \mathbb{E}[\mathcal{R}] - 0.5 \cdot (\mathbb{E}[\mathcal{R}] - \min(\mathcal{R})) \quad (3.7)$$

3.7.1 Infraestructura Computacional de Alto Rendimiento y Orquestación Distribuida

Dada la elevada dimensionalidad del espacio de búsqueda y el costo computacional intrínseco de la simulación microscópica a paso fino en SUMO, la exploración hiperparamétrica y el entrenamiento final de los agentes MARL requirieron el despliegue de una infraestructura de cómputo de alto rendimiento (HPC).

Las simulaciones se ejecutaron en los clusters de supercómputo **Toko** y **Mendieta** del Centro de Cómputo de Alto Rendimiento. La orquestación y asignación de recursos fue gestionada mediante el sistema de colas de trabajos SLURM (*Simple Linux Utility for Resource Management*), coordinando la ejecución paralela de múltiples instancias de simulación mediante los siguientes mecanismos:

- **Asignación Dinámica de Recursos:** Cada trabajo de simulación se dimensionó asignando núcleos de procesamiento (CPU) e intervalos de memoria RAM dedicados, optimizando la ejecución de múltiples procesos concurrentes de TraCI/`libsumo` y *workers* de Ray RLlib.
- **Gestión Eficiente de Entrada/Salida (I/O):** La simulación continua de redes viales genera un volumen crítico de eventos temporales por segundo. Para evitar cuellos de botella en el ancho de banda del sistema de archivos distribuido compartido en red (NFS), las simulaciones y la recolección de trayectorias se ejecutaron en las unidades de almacenamiento volátil local de alta velocidad de cada nodo de cómputo. Al finalizar cada prueba episódica, un proceso de sincronización transfiere únicamente las métricas agregadas y los puntos de control (*checkpoints*) consolidados.
- **Escalabilidad de la Exploración:** Esta arquitectura permitió procesar cientos de miles de iteraciones de entrenamiento y evaluar de forma automatizada las distintas fases de exploración Bayesiana (Optuna TPE) y la evaluación final en semillas no vistas.

—

3.8 Diseño de Experimentos de Evaluación de Rendimiento

El protocolo de evaluación final de los modelos se rigió por las siguientes condiciones:

1. **Semillas No Vistas (OOD):** Evaluación sobre $N = 20$ semillas aleatorias independientes disjuntas del proceso de entrenamiento (101, 137, 179, ..., 563), habiéndose excluido la semilla 42 por uso previo en pruebas piloto.
2. **Pruebas de Hipótesis:** Comparación de distribuciones de demora W_{net} mediante la prueba no paramétrica de Wilcoxon y cuantificación del tamaño del efecto mediante $\hat{A}_{12}$ de Vargha-Delaney.
3. **Guardias de Validación (*Fail-Closed Guards*):** Descarte automático de políticas candidatas que presenten una degradación de throughput superior al 5%, un incremento en teletransportes o una regresión de equidad (*fairness*) mayor al 10% respecto al control de tiempo fijo.

Análisis y discusión de resultados

En este capítulo se presentan y analizan los resultados obtenidos de las ejecuciones de entrenamiento de los algoritmos de estudio mencionados. Con el objetivo de distinguir con claridad las etapas metodológicas del entrenamiento de los agentes, la sección se organiza en tres partes.

En primer lugar, se reportan los resultados del proceso de ajuste de hiperparámetros. En segundo lugar, se evalúan los modelos finales seleccionados en los escenarios planteados, desde redes sintéticas hasta el escenario ubicado en Ciudad de Mendoza. Finalmente, se desarrolla una discusión integradora de los hallazgos, sus implicancias y las principales limitaciones del estudio.

4.1 Resultados del ajuste de hiperparámetros

En esta sección se resume el proceso de ajuste de hiperparámetros y se justifica la configuración utilizada en la evaluación posterior de los modelos.

4.1.1 Diseño del proceso de ajuste

4.1.2 Resultados del ajuste y análisis de sensibilidad

4.1.3 Configuración seleccionada para la evaluación final

4.2 Resultados de los modelos finales

Esta sección reúne los resultados obtenidos con los modelos finales una vez fijada la configuración experimental. La exposición se organiza según el tipo de escenario analizado.

4.2.1 Criterios de evaluación y métricas de comparación

4.2.2 Desempeño en escenarios sintéticos (3×1 , 3×3 , 4×4 heterogéneo)

4.2.3 Desempeño en la red de carreteras de la Ciudad de Mendoza

4.3 Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

Conclusiones

Por último las conclusiones.

Bibliografía

- [Ach18] Achiam, Joshua: *Spinning Up in Deep Reinforcement Learning*, 2018. OpenAI. Disponible en: <https://spinningup.openai.com/>.
- [ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. <https://www.marl-book.com/>, Consultado en 2025.
- [ALBBW⁺18] Alvarez Lopez, Pablo, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner y Evamarie Wießner: *Microscopic Traffic Simulation using SUMO*. En *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, páginas 2575–2582. IEEE, 2018.
- [Ale] Alegre, Lucas N.: *SUMO-RL 1.4.5 documentation*. Available at: <https://lucasalegre.github.io/sumo-rl/>.
- [Any25] Anyscale: *Ray Documentation: Scalable Distributed Computing and Hyperparameter Tuning with Tune and RLlib*, 2025. Disponible en: <https://docs.ray.io/>.
- [AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.
- [APK03] Abdulhai, Baher, Rob Pringle y Grigoris J Karakoulas: *Reinforcement learning for true adaptive traffic signal control*. Journal of Transportation Engineering, 129(3):278–285, 2003.
- [AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*, 2022. <https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf>, Fuente: [7, 8] Destaca el "éxito limitado" de SARL y los enfoques independientes en redes.
- [ASY⁺19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takuya Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.
- [CWCL19] Chu, Tianshu, Jie Wang, Lara Codecà y Zhaojian Li: *Multi-agent deep reinforcement learning for large-scale traffic signal control*. IEEE Transactions on Intelligent Transportation Systems, 21(3):1086–1095, 2019.
- [CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.
- [DK11] Devlin, Sam y Daniel Kudenko: *Potential-Based Difference Rewards for Multiagent Reinforcement Learning*. Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS), páginas 1037–1044, 2011.
- [DLR25] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2025. <https://sumo.dlr.de/docs/>, Consultado en 2025.

- [dWGM⁺20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr *y cols.*: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.
- [Ecl25] Eclipse Foundation: *SUMO Documentation – Simulation of Urban MObility*, 2025. Disponible en: <https://sumo.dlr.de/docs/>.
- [ETAA13] El-Tantawy, Samah, Baher Abdulhai y Hossam Abdulbaqi: *Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (MARLIN-ATSC): methodology and large-scale application on downtown Toronto*. IEEE Transactions on Intelligent Transportation Systems, 14(3):1140–1150, 2013.
- [FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.
- [Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: <https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm>.
- [FH21] Feriani, Amal y Ekram Hossain: *Multi-Agent Reinforcement Learning: Key Differences from Single-Agent RL*. arXiv preprint arXiv:2508.07001, 2021. Fuente: [9] Define la no estacionariedad como una diferencia clave de MARL vs SARL.
- [FNF⁺17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.
- [HSE⁺19] Hessel, Matteo, Hubert Soyer, Lasse Espeholt, Joseph Modayil, Hado Sanchez, van Hasselt y David Silver: *Multi-task deep reinforcement learning with PopArt*. En *Proceedings of the AAAI Conference on Artificial Intelligence*, volumen 33, páginas 3794–3801, 2019.
- [KCW⁺21] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. arXiv preprint arXiv:2109.11251, 2021.
- [KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.
- [LJR⁺20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzvi, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning*. En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.
- [LLN⁺18a] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.
- [LLN⁺18b] Liaw, Richard, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez y Ion Stoica: *Tune: A research platform for distributed model selection and training*. arXiv preprint arXiv:1807.05118, 2018.

- [LLW16] Li, Li, Yisheng Lv y Fei Yue Wang: *Traffic signal timing via deep reinforcement learning*. IEEE/CAA Journal of Automatica Sinica, 3(3):247–254, 2016.
- [LWT⁺17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.
- [MKS⁺15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. Nature, 518(7540):529–533, 2015. <https://doi.org/10.1038/nature14236>.
- [ne18] especificados, Autores no: *Traffic congestion cost Americans over \$160 billion in lost productivity*. Citado en arXiv:2406.13693 (Reporte original de UN 2018), 2018. Fuente: [1] Costos de congestión y emisiones de CO₂.
- [ne25] especificados, Autores no: *A Comprehensive Review of Reinforcement-Learning-Based Traffic Signal Control Methods*. Infrastructures, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.
- [Opt25] Optuna Development Team: *Optuna: Flexible Automated Hyperparameter Optimization Framework*, 2025. Disponible en: <https://optuna.org/>.
- [PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.
- [RSS⁺18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning*. En *International conference on machine learning*, páginas 4295–4304. PMLR, 2018.
- [SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.
- [Sha53] Shapley, Lloyd S: *Stochastic games*. Proceedings of the National Academy of Sciences, 39(10):1095–1100, 1953.
- [SLA⁺15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.
- [SLG⁺17] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z Leibo, Karl Tuyls y cols.: *Value-decomposition networks for cooperative multi-agent learning*. arXiv preprint arXiv:1706.05296, 2017.
- [SML⁺16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.
- [SWD⁺17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.

- [TMK⁺17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. PloS one, 12(4):e0172395, 2017.
- [W⁺19] Wei, H. y cols.: *A survey on traffic signal control methods*, 2019. Available at: <https://arxiv.org/abs/1904.08117>.
- [WCZ⁺19] Wei, Hua, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu y Zhenhui Li: *PressLight: Learning max pressure control for traffic signal in automated vehicles*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 1290–1298, 2019.
- [Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.
- [WZK⁺23] Wang, Xihuai, Lingpan Zheng, Jakub Grudzien Kuba, Ruiqing Chen, Li Shen, Yidong Cao, Jun Wang y Yaodong Yang: *Order matters: Agent-by-agent policy optimization*. En *International Conference on Learning Representations (ICLR)*, 2023.
- [YVV⁺22] Yu, Chao, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.
- [ZKF⁺24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of Machine Learning Research, 25:1–67, 2024.
- [ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.